Métodos computacionales para Daudin & Delarue (2025)

Aplicación numérica de Neural ODEs de campo medio con regularización entrópica
Basado en *arXiv:2507.08486*

Daniel Torres Gonzalez
Universidad Complutense de Madrid

24 de abril de 2026

Índice

Introducción **3**

1. Solución explícita: aplicación directa del Teorema 1.4 **3**
1.1. Planteamiento del problema . . . 3
1.2. Sistema forward–backward y fórmula explícita . . . 4
1.3. Aproximar ν_t . . . 4
1.4. Discretizaciones . . . 5
1.4.1. Discretización del espacio de parámetros A . . . 5
1.4.2. Discretización temporal . . . 5
1.4.3. Discretización de γ_t^* por partículas . . . 5
1.5. Backward por el método de las características . . . 6
1.6. Aproximación de la fórmula explícita en el grid . . . 6
1.7. Coste discreto . . . 7
1.8. Algoritmo: iteración de Picard con punto fijo . . . 7
1.9. Continuación en ε (warm-start) . . . 8
1.10. Experimentos . . . 8
1.10.1. Configuración común . . . 8
1.10.2. Datasets . . . 8
1.10.3. Resultados . . . 9
1.11. Alternativas numéricas para subir el techo dimensional . . . 10

2. Regularizadores sobre `make_moons`: SGLD, pSGLD, MMD^2 y Sinkhorn **12**
2.1. Obstrucción KL$= +\infty$ y motivación . . . 12
2.2. SGLD vanilla y por qué no basta . . . 13
2.3. pSGLD (Li et al. 2016) . . . 13
2.4. Regularizador MMD^2 . . . 13
2.5. Divergencia de Sinkhorn debiased . . . 14
2.6. Comparación experimental . . . 15
2.7. Solución explícita en `make_moons`: extensión n-D . . . 16

3. Regresión multivariada *sin* PCA: California Housing en $d_1 = 8$ **18**
3.1. Setup . . . 18
3.2. Resultados . . . 19
3.3. Lectura . . . 19
3.4. Solución explícita: limitación dimensional . . . 21

4. *Bridge*: ¿cuál de los regularizadores reproduce la distribución de Gibbs del paper? **22**

5. Aplicación económica: predicción de ingresos sobre `ACSIncome` **25**

5.1. El dataset `ACSIncome` 26
5.2. Setting experimental 26
5.3. Resultados *(A)* —- solver explícito $d_1 = 2$ 26
5.4. Resultados *(B)* —- proxies paramétricos $d_1 = 4$ 27
5.5. Resultados *(C)* —- bridge sobre datos reales 28
5.6. Otras posibles formulaciones económicas 29

6. Trabajo futuro **29**

Introducción

Este documento describe en detalle los métodos numéricos implementados para la aplicación computacional del marco de **Daudin & Delarue** (2025) sobre *Neural ODEs de campo medio con regularización entrópica*. Cada sección corresponde a un método distinto de aproximar o calcular la trayectoria óptima $\boldsymbol{\nu}^* = (\nu_t^*)_{t\in[t_0,T]}$ del control:

1. **Solución explícita** (Sección 1): aplicación directa de la fórmula cerrada del Teorema 1.4 del paper sobre un *grid* discreto del espacio de parámetros A. Es la implementación más fiel posible al marco teórico, sin aproximaciones más allá de la discretización espacio-temporal.

2. **Regularizadores paramétricos** (Sección 2): SGLD vanilla, pSGLD, MMD^2 y divergencia de Sinkhorn como sustitutos prácticos del término entrópico (que es $+\infty$ en la representación por M-partículas). Comparación sobre `make_moons` y, en Sección 3, sobre California Housing en $d_1 = 8$.

3. **Bridge interpretativo** (Sección 4): comparación cuantitativa entre la ν_T^* exacta del solver explícito y la nube empírica de neuronas de cada regularizador, con MMD^2 y los dos primeros momentos como métricas.

4. **Aplicación económica** (Sección 5): replicación del pipeline completo (solver explícito + tres proxies + bridge) sobre `ACSIncome`, un dataset demográfico real (American Community Survey, California 2018).

5. **Trabajo futuro** (Sección 6): cuatro líneas naturales que el bridge habilita —tasas cuantitativas de propagación de caos, bridge dinámico, regularizador "DD-aware", tests de robustez cross-dataset.

1. Solución explícita: aplicación directa del Teorema 1.4

1.1. Planteamiento del problema

Recordamos el problema de control estudiado en Daudin & Delarue:

$$\min_{\boldsymbol{\nu}} \; J\big((t_0,\gamma_0),\boldsymbol{\nu}\big) \;=\; \int_{\mathbb{R}^{d_1}\times\mathbb{R}^{d_2}} L(x,y)\;\mathrm{d}\gamma_T(x,y) \;+\; \varepsilon\int_{t_0}^{T} \mathcal{E}(\nu_t \,\|\, \nu^\infty)\;\mathrm{d}t, \tag{1}$$

sujeto a la ecuación de continuidad

$$\partial_t\gamma_t + \mathrm{div}_x\left(\int_A b(x,a)\;\mathrm{d}\nu_t(a)\;\gamma_t\right) = 0, \qquad \gamma_{t_0} = \gamma_0. \tag{2}$$

El control $\boldsymbol{\nu} = (\nu_t)_{t\in[t_0,T]}$ es una trayectoria en el espacio de medidas de probabilidad sobre el conjunto de parámetros $A \subset \mathbb{R}^{d'}$. El término entrópico $\mathcal{E}(\nu_t \,\|\, \nu^\infty)$ es la divergencia de Kullback–Leibler de ν_t respecto a la medida prior $\nu^\infty(a) \propto \exp(-\ell(a))$, con potencial

$$\ell(a) = c_1\,|a|^4 + c_2\,|a|^2\,, \qquad c_1, c_2 > 0 \tag{3}$$

(necesariamente con crecimiento al menos cuártico para garantizar la desigualdad log-Sobolev y por tanto la condición Polyak–Łojasiewicz, véase Sec. 1.2 del paper).

Para máxima fidelidad al **Ejemplo 1.3** del paper, fijamos los datos:

$$d_1 = d_2 = 1, \qquad L(x,y) = \tfrac{1}{2}(x-y)^2, \qquad b(x,a) = \sigma(a_1x + a_2)\,a_0, \tag{4}$$

con $\sigma = \tanh$ y $a = (a_0, a_1, a_2) \in \mathbb{R}^3$, esto es, $d' = 2d_1 + 1 = 3$.

Observación 1.1 (Por qué $d_1 = d_2 = 1$)**.** La elección $d_1 = d_2 = 1$ no es estética: las hipótesis del paper sobre la pérdida cuadrática requieren $d_1 = d_2$, y el grid sobre $A = \mathbb{R}^{d'}$ con $d' = 2d_1 + 1$ se vuelve inviable más allá de $d_1 = 2$. Para $d_1 = 1$: $d' = 3$, grid 10^3 totalmente manejable.

1.2. Sistema forward–backward y fórmula explícita

El Teorema 1.4 del paper caracteriza cualquier minimizador $\boldsymbol{\nu}^* \in \mathcal{A}(t_0)$ del funcional (1) mediante la fórmula de Gibbs explícita

$$\boxed{\nu_t^*(a) \;=\; \frac{1}{Z_t^*}\exp\!\left(-\ell(a) \;-\; \frac{1}{\varepsilon}\int_{\mathbb{R}^{d_1}\times\mathbb{R}^{d_2}} b(x,a)\cdot\nabla_x u_t^*(x,y)\;\mathrm{d}\gamma_t^*(x,y)\right)} \tag{5}$$

donde Z_t^* es la constante de normalización, y $(\boldsymbol{u}^*, \boldsymbol{\gamma}^*)$ resuelven el sistema forward–backward acoplado:

$$\begin{cases} \partial_t\gamma_t^* + \mathrm{div}_x\big(b(x,\nu_t^*)\,\gamma_t^*\big) = 0, & \gamma_{t_0}^* = \gamma_0, \\ -\partial_t u_t^*(x,y) - b(x,\nu_t^*)\cdot\nabla_x u_t^*(x,y) = 0, & u_T^*(x,y) = L(x,y). \end{cases} \tag{6}$$

Aquí hemos abreviado $b(x,\nu_t) := \int_A b(x,a)\;\mathrm{d}\nu_t(a)$.

Observación 1.2 (ν_t^* identificada con su densidad)**.** La fórmula (5) expresa ν_t^* por su *densidad* respecto a la medida de Lebesgue en A, no como medida singular. Esto es legítimo en el setting del Teorema 1.4: bajo las hipótesis del paper (potencial ℓ supercoercivo, $\varepsilon > 0$), ν_t^* es absolutamente continua respecto a ν^∞, y por tanto respecto a Lebesgue, para todo $t \in [t_0, T]$. En el resto del documento usamos indistintamente la medida y su densidad cuando no hay riesgo de confusión: $\nu_t^*(a)$ denota el valor puntual de la densidad y $\int f(a)\,d\nu_t^*(a)$ la integral contra esa misma densidad. La distinción reaparece sólo al comparar ν_t^* con sus aproximaciones *paramétricas* $\frac{1}{M}\sum_m \delta_{\theta^m(t)}$, que sí son singulares respecto a Lebesgue (cf. Sección 2.1).

El sistema (6)–(5) es de tipo *Mean-Field Game*: la dinámica forward de las features depende del control ν_t^*, que a su vez se determina por la dinámica backward del adjunto u_t^*, evaluado contra la misma γ_t^*. Resolverlo de forma exacta es generalmente imposible; la idea de nuestra aproximación es:

1. Discretizar el espacio A en un *grid* finito $\{a^l\}_{l=1}^M$ para que ν_t^* se reduzca a un vector de probabilidades $\{\nu_t^*(a^l)\}_{l=1}^M$.
2. Discretizar γ_t^* por N partículas $(X_i(t), y_i)$.
3. Resolver (6)–(5) por una iteración de tipo Picard (punto fijo).
4. Calcular el adjunto u^* por el método de las características, evitando integrar la EDP backward explícitamente.

1.3. Aproximar ν_t

Las dos formas de aproximar ν_t. En la práctica ν_t no se representa exactamente; se aproxima de una de dos maneras complementarias:

- **Aproximación *paramétrica* con M partículas (Tareas 2 y 3, Sec. 2 y posteriores).** Se fijan M neuronas $\{\theta^m\}_{m=1}^M \subset A$ y se identifica $\nu_t \approx \frac{1}{M}\sum_{m=1}^M \delta_{\theta^m(t)}$. **Las posiciones θ^m se aprenden** (descenso por gradiente, pSGLD...) y se mueven libremente por A. La red es la familiar `MeanFieldResNet`: "M neuronas con parámetros entrenables".
- **Aproximación *explícita* sobre un grid (esta sección).** Se fijan M neuronas *candidatas* $\{a^l\}_{l=1}^M$ en posiciones *predeterminadas* (puntos de un grid regular sobre A) y se identifica $\nu_t \approx \sum_{l=1}^M \nu_t(a^l)\,\delta_{a^l}$. **Las posiciones a^l no se mueven**; lo único que se optimiza es el *vector de probabilidades* $(\nu_t(a^l))_{l=1}^M$ que dice "cuánto pesa cada neurona del catálogo en el tiempo t".

Por qué el grid en este enfoque. La fórmula del Teorema 1.4 (ec. 5) caracteriza ν_t^* como una distribución de Gibbs:

$$\nu_t^*(a) \;\propto\; \exp\Big(-\ell(a) - \tfrac{1}{\varepsilon}\mathbb{E}_{\gamma_t^*}\big[b(x,a)\cdot\nabla_x u_t^*\big]\Big).$$

Para *evaluar* esta densidad y para *normalizarla* ($\int_A \nu_t^* = 1$) hacen falta integrales sobre A. El grid es la herramienta más simple para hacerlas: se reemplaza $\int_A \nu_t^*(a)\,da$ por una suma sobre los puntos del grid y ν_t^* pasa a ser un *vector de longitud* M, manejable y diferenciable.

Limitaciones del grid. El grid uniforme escala como $M = M_{\text{pd}}^{2d_1+1}$ puntos por dimensión. Para $d_1 = 1$ ($A = \mathbb{R}^3$) basta $M_{\text{pd}} = 10$ y $M = 1\,000$; para $d_1 = 8$ ($A = \mathbb{R}^{17}$) ya con $M_{\text{pd}} = 3$ se llegaría a $3^{17} \approx 1{,}3 \cdot 10^8$ puntos: *maldición de la dimensionalidad.* Por eso el enfoque explícito (esta sección) sólo es viable en dimensión baja, mientras que la aproximación paramétrica con M partículas (siguientes secciones) escala bien.

1.4. Discretizaciones

1.4.1. Discretización del espacio de parámetros A

Construimos un grid uniforme cartesiano sobre $[-R, R]^3 \subset A$:

$$\{a^l\}_{l=1}^M \;:=\; \big\{\big(g_{i_0},\, g_{i_1},\, g_{i_2}\big) \;:\; i_0, i_1, i_2 \in \{1,\dots,M_{\text{dim}}\}\big\}, \quad g_i = -R + \tfrac{2R\,(i-1)}{M_{\text{dim}}-1}, \tag{7}$$

con $M = M_{\text{dim}}^3$ puntos. En todos los experimentos usamos $R = 2{,}5$ y $M_{\text{dim}} = 10$, es decir, $M = 1000$.

La medida prior ν^∞, restringida al grid y normalizada, es

$$\nu^\infty(a^l) \;=\; \frac{\exp(-\ell(a^l))}{\sum_{l'=1}^M \exp(-\ell(a^{l'}))}. \tag{8}$$

La elección $R = 2{,}5$ cubre más del 99 % de la masa de la prior continua, ya que $\ell(a) \geq c_1\,|a|^4$ decae rápidamente fuera de la bola unidad.

1.4.2. Discretización temporal

Particionamos $[t_0, T] = [0, T]$ en K subintervalos de tamaño $\Delta t = T/K$:

$$t_k = k\,\Delta t, \quad k = 0, 1, \dots, K. \tag{9}$$

La trayectoria de la medida ν_t^* se almacena como una matriz $\boldsymbol{\nu} \in \mathbb{R}^{(K+1)\times M}$ con $\boldsymbol{\nu}_{k,l} \approx \nu_{t_k}^*(a^l)$.

1.4.3. Discretización de γ_t^* por partículas

Sustituimos la medida γ_t por la suma empírica sobre N partículas con etiquetas fijas:

$$\gamma_t^{(N)} = \frac{1}{N}\sum_{i=1}^N \delta_{(X_i(t),\,y_i)}, \quad X_i(0) = x_i, \;\; i = 1, \dots, N. \tag{10}$$

Cada partícula evoluciona según la ODE

$$\dot{X}_i(t) \;=\; \int_A b(X_i(t), a)\;\mathrm{d}\nu_t(a) \;\approx\; \sum_{l=1}^M b(X_i(t), a^l)\,\nu_t(a^l). \tag{11}$$

1.5. Backward por el método de las características

La ecuación backward del sistema (6),

$$-\partial_t u_t - b(x, \nu_t) \cdot \nabla_x u_t = 0, \qquad u_T = L, \tag{12}$$

es una *ecuación de transporte* cuyas características son exactamente las trayectorias de la dinámica forward. Esto permite obtener u^* *sin integrar la EDP*, simplemente leyendo L al tiempo final a lo largo del flujo:

$$u_t^*(x, y) \;=\; L\big(X^{t,x}(T),\, y\big), \tag{13}$$

donde $X^{t,x}(s)$ denota la solución de (11) en tiempo s con condición inicial x en tiempo t.

Gradiente espacial $\nabla_x u_t^*$. Para evaluar la fórmula (5) necesitamos $\nabla_x u_t^*(X_i(t), y_i)$. Por la regla de la cadena aplicada a (13), con $L(x, y) = \frac{1}{2}(x - y)^2$:

$$\nabla_x u_t^*(x, y) \;=\; \big(X^{t,x}(T) - y\big) \cdot \frac{\partial X^{t,x}(T)}{\partial x}. \tag{14}$$

Sensibilidad de la trayectoria. Para cada partícula i, denotemos

$$\Phi_i(t) \;:=\; \frac{\partial X_i(t)}{\partial X_i(0)}, \tag{15}$$

es decir, la sensibilidad de la trayectoria respecto a su condición inicial (usamos la letra Φ, habitual para la *matriz fundamental* en EDOs lineales, para no colisionar con el funcional de coste J de (1)). Esta función satisface la *ecuación variacional*:

$$\dot{\Phi}_i(t) \;=\; \frac{\partial}{\partial x}\left[\int_A b(X_i(t), a)\; \mathrm{d}\nu_t(a)\right]_{x=X_i(t)} \cdot \Phi_i(t), \qquad \Phi_i(0) = 1. \tag{16}$$

En el caso $d_1 = 1$, $\Phi_i(t)$ es *escalar*, lo que simplifica mucho la implementación. Por la propiedad del cociclo del flujo:

$$\frac{\partial X^{t,X_i(t)}(T)}{\partial x} \;=\; \frac{\Phi_i(T)}{\Phi_i(t)}. \tag{17}$$

Así, sustituyendo en (14) con $x = X_i(t)$, $y = y_i$, obtenemos la fórmula computable:

$$\boxed{\nabla_x u_t^*\big(X_i(t),\, y_i\big) \;=\; \big(X_i(T) - y_i\big) \cdot \frac{\Phi_i(T)}{\Phi_i(t)}.} \tag{18}$$

Esta identidad es la pieza clave que evita resolver la EDP backward por diferencias finitas o por método adjunto: basta integrar *conjuntamente* (X_i, Φ_i) por (11) y (16) durante el paso forward y, al terminar, leer (18) en cualquier t_k deseado.

1.6. Aproximación de la fórmula explícita en el grid

Sustituyendo la aproximación empírica $\gamma_t^* \approx \gamma_t^{(N)}$ y la identidad (18) en (5), obtenemos la versión discretizada de la fórmula de Gibbs sobre el grid:

$$\log \nu_{t_k}^*(a^l) \;=\; -\ell(a^l) \;-\; \frac{1}{\varepsilon N}\sum_{i=1}^{N} b\big(X_i(t_k), a^l\big) \cdot \big(X_i(T) - y_i\big)\, \frac{\Phi_i(T)}{\Phi_i(t_k)} \;-\; \log Z_{t_k}^{(M)}, \tag{19}$$

con $Z_{t_k}^{(M)}$ la constante de normalización en el grid. Numéricamente calculamos el lado derecho con `logsumexp` para evitar `overflow`.

1.7. Coste discreto

El funcional (1), evaluado sobre las discretizaciones, se escribe

$$J^{(N,M,K)}(\boldsymbol{\nu}) \;=\; \underbrace{\frac{1}{N}\sum_{i=1}^{N}\tfrac{1}{2}\big(X_i(T)-y_i\big)^2}_{\text{término terminal}} \;+\; \varepsilon\,\Delta t\,\underbrace{\sum_{k=0}^{K}\mathrm{KL}\big(\boldsymbol{\nu}_k \,\|\, \nu^\infty\big)}_{\text{término entrópico}}, \tag{20}$$

donde la KL discreta es

$$\mathrm{KL}(\boldsymbol{\nu}_k \,\|\, \nu^\infty) \;=\; \sum_{l=1}^{M}\boldsymbol{\nu}_{k,l}\,\log\frac{\boldsymbol{\nu}_{k,l}}{\nu^\infty(a^l)}, \tag{21}$$

con la convención usual $0\log 0 = 0$.

Observación 1.3 (KL finita)**.** Una de las motivaciones principales de la discretización por grid es que la divergencia $\mathrm{KL}(\nu_{t_k} \,\|\, \nu^\infty)$ es *finita*: ambas medidas son discretas sobre el mismo soporte finito $\{a^l\}$. En contraste, las aproximaciones por partículas $\nu_t^{(M)} = \frac{1}{M}\sum_m \delta_{a^m}$ producen $\mathrm{KL}(\nu_t^{(M)} \,\|\, \nu^\infty) = +\infty$ porque $\nu_t^{(M)}$ es discreta y ν^∞ continua. Esta es la limitación que motiva los métodos sustitutivos (pSGLD, MMD, Sinkhorn) de las secciones siguientes.

1.8. Algoritmo: iteración de Picard con punto fijo

El sistema (6)–(5) es un *problema de punto fijo*: ν_t^* depende de (γ_t^*, u_t^*), que a su vez dependen de ν_t^*. Iteramos:

Algorithm 1 Picard fixed-point para la solución explícita

1: **Entrada:** dataset $\{(x_i, y_i)\}_{i=1}^N$, grid $\{a^l\}_{l=1}^M$, hiperparámetros $T, K, \varepsilon, c_1, c_2$, factor de damping $\omega \in (0,1]$, tolerancia τ, ν_{init} opcional.
2: Calcular $\nu^\infty(a^l)$ por (8).
3: Inicializar $\boldsymbol{\nu}_k^{(0)} \leftarrow \nu^\infty$ para todo k (o ν_{init} si se proporciona).
4: **for** $j = 0, 1, 2, \ldots$ hasta $\max_k \left\|\boldsymbol{\nu}_k^{(j+1)} - \boldsymbol{\nu}_k^{(j)}\right\|_{\mathrm{TV}}/2 < \tau$ **do**
5: **Forward:** integrar (X_i, Φ_i) por RK4 sobre $[0,T]$ con control $\boldsymbol{\nu}^{(j)}$, usando (11) y (16).
6: **Update Gibbs:** para cada $k = 0, \ldots, K$ y cada $l = 1, \ldots, M$, calcular $\tilde{\boldsymbol{\nu}}_{k,l}$ por (19) usando $X_i(t_k)$, $\Phi_i(t_k)$, $\Phi_i(T)$, $X_i(T)$.
7: **Damping:** $\boldsymbol{\nu}_k^{(j+1)} \leftarrow (1-\omega)\,\boldsymbol{\nu}_k^{(j)} + \omega\,\tilde{\boldsymbol{\nu}}_k$, renormalizando.
8: **end for**
9: **Salida:** $\boldsymbol{\nu}^{(j)}$ (ley óptima discreta), $\{X_i(t_k)\}_{i,k}$ (trayectorias), $\{\Phi_i(t_k)\}_{i,k}$ (Jacobianos del flujo), y $J^{(N,M,K)}(\boldsymbol{\nu}^{(j)})$ (coste discreto evaluado en la última iteración).

Observación 1.4 (Damping)**.** El factor $\omega \in (0,1]$ es necesario para estabilidad. Sin damping ($\omega = 1$), Picard puede oscilar entre dos puntos lejanos del óptimo, especialmente cuando ε es pequeño y la fórmula de Gibbs produce concentraciones agresivas. Empíricamente $\omega = 0{,}2$–$0{,}3$ funciona bien para el rango de ε explorado.

Observación 1.5 (Integrador RK4 con ν interpolada)**.** El paso RK4 estándar evalúa el campo en cuatro instantes intermedios del subintervalo $[t_k, t_{k+1}]$. Como ν_t está discretizada sólo en los nodos t_k, interpolamos linealmente entre $\boldsymbol{\nu}_k$ y $\boldsymbol{\nu}_{k+1}$ para los sub-pasos intermedios. Esta es una decisión de implementación, no del paper.

1.9. Continuación en ε (warm-start)

Para valores pequeños de ε, la fórmula de Gibbs (19) concentra mucha masa en pocos puntos del grid, y Picard *frío* (inicializado en ν^∞) cicla sin converger. Implementamos una estrategia de *continuación* que calienta progresivamente el problema:

Algorithm 2 Continuación en ε

1: **Entrada:** secuencia decreciente $\varepsilon_1 > \varepsilon_2 > \ldots > \varepsilon_S$.
2: $\boldsymbol{\nu}_{\text{warm}} \leftarrow \nu^\infty$.
3: **for** $s = 1, 2, \ldots, S$ **do**
4: Resolver Picard (Algoritmo 1) con $\varepsilon = \varepsilon_s$ y $\nu_{\text{init}} = \boldsymbol{\nu}_{\text{warm}}$.
5: $\boldsymbol{\nu}_{\text{warm}} \leftarrow$ solución devuelta por Picard.
6: **end for**

Empíricamente, esta estrategia extiende el rango estable de ε desde $\varepsilon \geq 0{,}05$ (frío) hasta $\varepsilon \geq 0{,}02$ (con continuación) en los datasets considerados.

1.10. Experimentos

1.10.1. Configuración común

Hiperparámetro	Valor
Dimensiones	$d_1 = d_2 = 1$, $d' = 3$
Pérdida	$L(x, y) = \frac{1}{2}(x - y)^2$
Activación	$\sigma = \tanh$
Potencial	$\ell(a) = c_1 \lvert a\rvert^4 + c_2 \lvert a\rvert^2$, $(c_1, c_2) = (0{,}05,\ 0{,}5)$
Grid	$[-2{,}5,\ 2{,}5]^3$, $M_{\text{dim}} = 10 \Rightarrow M = 1000$
Horizonte	$T = 1$, $K = 20$, $\Delta t = 0{,}05$
Integrador	RK4 con ν interpolada linealmente
Damping	$\omega = 0{,}2$–$0{,}3$
Tolerancia	$\tau = 10^{-5}$ en $\lVert\cdot\rVert_{\text{TV}}$
Continuación	$\varepsilon \in \{0{,}5,\ 0{,}2,\ 0{,}1,\ 0{,}05,\ 0{,}02\}$

Cuadro 1: Configuración por defecto del solver explícito.

1.10.2. Datasets

Usamos dos datasets reales 1D→1D, ambos cargados de `vincentarelbundock.github.io/Rdatasets`:

- **Old Faithful** (`faithful.csv`): $N = 272$ observaciones del géiser de Yellowstone, con $x =$ duración de la erupción e $y =$ tiempo de espera hasta la siguiente. Correlación $\rho \approx 0{,}90$, relación esencialmente lineal.
- **trees** (`trees.csv`): $N = 31$ árboles de cerezo, con $x =$ *Girth* (diámetro del tronco) e $y =$ *Volume* (volumen de madera). Correlación $\rho \approx 0{,}97$, relación *no lineal* de tipo $V \propto G^2$ (ley geométrica del cilindro).

Las features y etiquetas se estandarizan independientemente ($\mu = 0$, $\sigma = 1$) antes del entrenamiento.

Como referencia teórica, comparamos contra el ajuste OLS lineal sin intercepto (válido tras estandarizar):

$$\hat{a}_{\text{OLS}} = \frac{\sum_i x_i y_i}{\sum_i x_i^2}, \qquad \text{MSE}/2 = \frac{1}{2N} \sum_i (\hat{a}_{\text{OLS}} x_i - y_i)^2. \tag{22}$$

Esta es la *cota inferior* para problemas con relación exclusivamente lineal y *cota superior alcanzable* cuando la mean-field ResNet sólo recupera la dependencia lineal.

1.10.3. Resultados

ε	iter.	convergido	J	término L	disp. media
Old Faithful (OLS lineal: MSE/2 = 0,0943)					
0,500	23	sí	0,0983	0,0976	0,017
0,200	22	sí	0,0975	0,0964	0,032
0,100	23	sí	0,0967	0,0954	0,046
0,050	26	sí	0,0959	0,0946	0,060
0,020	29	sí	0,0948	**0,0936**	0,074
trees (OLS lineal: MSE/2 = 0,0323)					
0,500	23	sí	0,0326	0,0323	0,007
0,200	24	sí	0,0323	0,0318	0,015
0,100	26	sí	0,0319	0,0312	0,022
0,050	28	sí	0,0314	0,0304	0,031
0,020	30	sí	0,0303	**0,0285**	0,042

Cuadro 2: Resultados de la solución explícita por continuación en ε. "Disp. media" es $\frac{1}{N}\sum_i |X_i(T) - X_i(0)|$, una medida del trabajo realizado por el flujo.

Lectura.

- En **Old Faithful**, el término L converge a 0,0936, ligeramente *por debajo* de OLS (0,0943). El gap es marginal: el dataset es esencialmente lineal, y la mean-field ResNet sólo puede aprovechar muy poca no linealidad.
- En **trees**, el término L desciende hasta 0,0285 (un 12 % por debajo de OLS = 0,0323). Esto demuestra que la mean-field ResNet *captura* la no linealidad $V \propto G^2$.
- En ambos casos, la desviación entre el coste total J y el término L corresponde exactamente al término entrópico $\varepsilon\,\Delta t \sum_k \mathrm{KL}$, como debe ser por construcción.

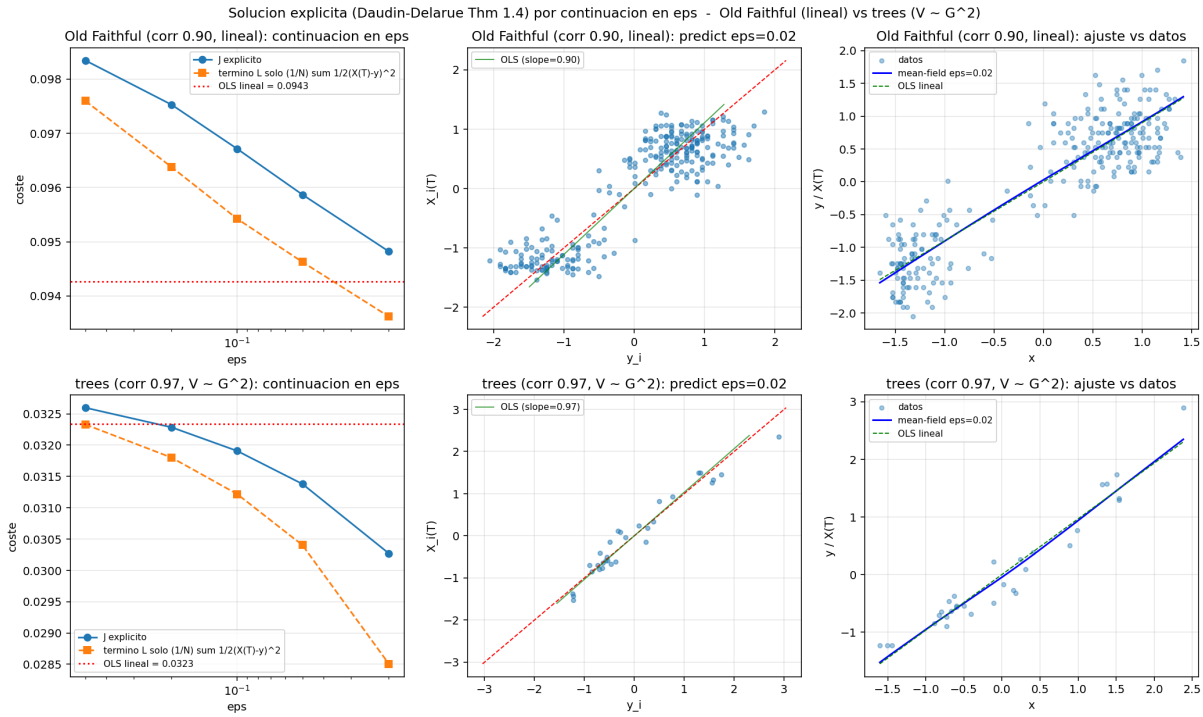

Figura 1: Solución explícita por continuación en ε. *Fila superior:* Old Faithful (lineal). *Fila inferior:* trees ($V \propto G^2$). *Columna 1:* cost vs ε. *Columna 2:* predicciones $X_i(T)$ contra y_i para el menor ε convergido. *Columna 3:* ajuste mean-field (azul) sobre los datos crudos junto con el OLS lineal (verde discontinuo); en *trees* se observa la curvatura.

1.11. Alternativas numéricas para subir el techo dimensional

El cuello de botella del solver explícito es la cuadratura del grid tensorial: $M = M_{\text{pd}}^{2d_1+1}$ celdas. En $d_1 = 1$ caben en memoria; en $d_1 = 2$ con $M_{\text{pd}} = 4$ ya estamos en $M = 1024$; en $d_1 = 8$ (California Housing completo) sería $M_{\text{pd}}^{17} \geq 10^8$ con $M_{\text{pd}} = 3$. Tres familias de cuadratura podrían subir este techo de $d_1 \leq 2$ actual a $d_1 \approx 4$–6, sin alterar la estructura del solver Picard:

QMC + importance sampling (implementado). Reemplazamos el grid tensorial por un *sample* Sobol *scrambled* de N puntos en $[0,1]^{2d_1+1}$, transformados al espacio de parámetros vía CDF inverso de una Gaussiana proposal $q \sim \mathcal{N}(0, \sigma^2 I)$. El peso de *importance sampling* es entonces $\log \nu^\infty(a) = -\ell(a) - \log q(a)$ (hasta normalización), y el resto del solver Picard queda inalterado: solo cambia la elección de los puntos de soporte. Comparamos sobre `make_moons` ($d_1 = 2$, $\dim A = 5$) tres familias en $\varepsilon = 0{,}05$:

método	M	J	term L	tiempo (s)
grid tensorial	32	0,338	0,265	12,4
grid tensorial	243	0,551	0,240	45,4
grid tensorial	1024	0,533	0,332	125,9
QMC Sobol scrambled	32	0,592	0,462	4,5
QMC Sobol scrambled	**256**	**0,510**	**0,339**	**29,8**
QMC Sobol scrambled	1024	0,515	0,324	130,9
MC iid (mismo proposal)	32	0,547	0,405	7,7
MC iid (mismo proposal)	256	0,527	0,320	52,3
MC iid (mismo proposal)	1024	0,506	0,320	138,2

Cuadro 3: Comparativa de cuadraturas para el solver explícito. **Hallazgo clave**: QMC con $M = 256$ alcanza el mismo término terminal $L \approx 0{,}34$ que el grid tensorial con $M = 1\,024$, pero con 4,2× menos tiempo. La diferencia entre QMC y MC iid es marginal en este régimen ($\dim A = 5$), lo que sugiere dimensión efectiva moderada del integrando. El valor anómalamente bajo de J y L del grid $M = 32$ es un artefacto del soporte minúsculo (la entropía discreta penaliza poco), no una mejora real, como confirma el hecho de que esa configuración no converge en el cap de iteraciones.

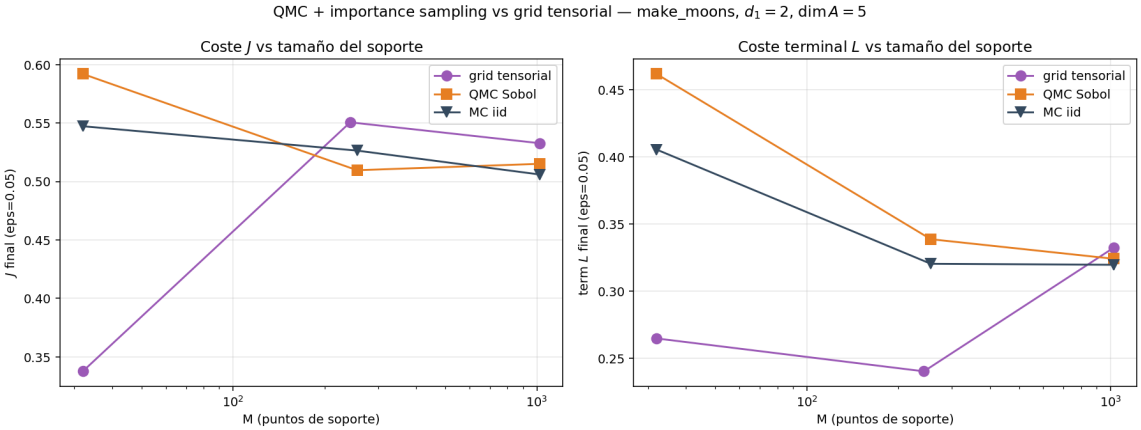

Figura 2: Coste J y término terminal L vs tamaño del soporte M (escala log). Para $M \geq 256$ las tres familias convergen al mismo valor; QMC y MC son indistinguibles en este régimen, pero ambos baten al grid tensorial en eficiencia *tiempo/calidad*.

Lectura: en $d_1 = 2$ el grid sigue siendo viable y la ganancia es modesta ($\sim 4\times$ tiempo). El interés real está en $d_1 \geq 3$, donde la cuadratura tensorial empieza a no caber en memoria pero N puede mantenerse fijo en QMC/MC — esto vuelve viables comparaciones cuantitativas en $d_1 \in \{3, 4, 5\}$ que el grid no permite.

Otras alternativas (no implementadas). Dos familias adicionales podrían subir más el techo dimensional:

- **Sparse grids (Smolyak)**. Escalan $\mathcal{O}(N\,(\log N)^{d-1})$ en lugar de $\mathcal{O}(N^d)$. Cada iteración Picard cambia φ, por lo que conviene combinar Smolyak con refinamiento dimensional adaptativo (*combination technique*). Esfuerzo medio, librerías estándar (Tasmanian, sparseSpACE).
- **Tensor trains (TT-cross)**. La estructura $b(x, a) = \tanh(a_1 \cdot x + a_2) \cdot a_0$ separa a_0 (entra linealmente) de (a_1, a_2) (entran por tanh), lo que *sugiere* rango TT bajo. Si se confirma,

el coste pasa a $\mathcal{O}(d \cdot r^2 \cdot n)$ y $d_1 = 8$ se vuelve viable. El coste de $\varphi(a) = \mathbb{E}_\gamma[b \cdot \nabla_x u]$ introduce acoplamiento adicional que podría degradar el rango; el riesgo es alto, el *payoff* potencialmente transformador.

Ninguna de estas alternativas vuelve competitivo al solver explícito frente al método paramétrico de la Sección 2: cada iteración Picard reconstruye el integrando, mientras que el optimizador paramétrico aprovecha vectorización GPU sobre mini-batches. El interés conceptual –comparar ν_t^* exacta vs. histogramas empíricos en dimensiones moderadas– es el que motiva todo el bloque, y se materializa parcialmente en la Sección 4.

2. Regularizadores sobre `make_moons`: SGLD, pSGLD, MMD^2 y Sinkhorn

En la Sección 1 se aplicó la fórmula explícita del Teorema 1.4 a problemas $1 \to 1$ donde el coste terminal $L(x, y) = \frac{1}{2}(x - y)^2$ y la dimensión del estado coinciden con las del paper. En esta sección se aborda `make_moons` ($d_1 = 2$, etiquetas binarias) por dos vías complementarias:

(A) Aproximaciones *paramétricas* basadas en `MeanFieldResNet`, donde la medida ν_t se sustituye por M partículas (θ^m). La penalización $\varepsilon\,\mathcal{E}(\nu_t \| \nu^\infty)$ es estrictamente infinita en este régimen (Sección 2.1), de manera que se exploran cuatro *sustitutos* bien definidos: SGLD vanilla, pSGLD, MMD^2 y Sinkhorn.

(B) Resolución *explícita* (extensión n-D del algoritmo de la Sección 1) sobre `make_moons` con un *embedding* de etiquetas $y \in \{0, 1\} \mapsto y_{\text{emb}} \in \mathbb{R}^2$, lo que encaja exactamente en el Ejemplo 1.3 del paper ($d_1 = d_2 = 2$).

2.1. Obstrucción KL$= +\infty$ y motivación

La penalización entrópica del paper es

$$\mathcal{E}(\nu_t \| \nu^\infty) \;=\; \int \log \frac{d\nu_t}{d\nu^\infty}\, d\nu_t, \qquad \nu^\infty(da) \propto e^{-\ell(a)}\, da.$$

En la implementación con M partículas $\nu_t = \frac{1}{M}\sum_m \delta_{\theta^m(t)}$ se tiene $\nu_t \perp \nu^\infty$ (la primera es discreta y la segunda absolutamente continua), de modo que $\mathrm{KL}(\nu_t \| \nu^\infty) = +\infty$ y $\mathcal{E}$ no se puede evaluar literalmente. Se contemplan tres familias de sustitutos:

- **Energía supercoerciva** (sin término entrópico): $\mathcal{E}_{\text{aprox}}(\nu_t) = \frac{1}{N_p}\sum_j \ell(\theta_j)$. Equivale a regularización $L^2 + L^4$ sobre los pesos. *Pierde* el componente $-\mathcal{H}(\nu_t)$ de la entropía diferencial.

- **Ruido de Langevin** (SGLD/pSGLD): se inyecta $\sqrt{2\eta\varepsilon M_t}\,\xi$ tras cada paso del optimizador. La cadena (θ_t), con $\theta \in \mathbb{R}^{M(2d_1+1)}$ (los M pesos concatenados), tiene como medida estacionaria $\nu^*_{\text{SGLD}}(\theta) \propto \exp(-J(\theta)/\varepsilon)$. *No* es literalmente la ν_t^* del Teorema 1.4 (que vive en $\mathcal{A} = \mathbb{R}^{2d_1+1}$ y depende de t), sino su *análogo finito-dimensional* en la parametrización M-partículas: cuando $M \to \infty$, la medida empírica $\frac{1}{M}\sum \delta_{a_i}$ de un sample $\theta \sim \nu^*_{\text{SGLD}}$ tiende, formalmente, al objeto continuo del paper (propagación de caos).

- **Distancias entre medidas** (MMD^2, Sinkhorn): se sustituye $\mathcal{E}(\nu_t \| \nu^\infty)$ por una distancia bien definida entre ν_t (partículas) y muestras del prior ν^∞ obtenidas por ULA.[1]

[1] ULA (*Unadjusted Langevin Algorithm*) es la discretización Euler-Maruyama de la SDE de Langevin $d\theta_t = -\nabla U(\theta_t)\,dt + \sqrt{2}\,dW_t$: $\theta_{k+1} = \theta_k - \eta\,\nabla U(\theta_k) + \sqrt{2\eta}\,\xi_k$ con $\xi_k \sim \mathcal{N}(0, I)$. Su distribución estacionaria aproxima $\propto \exp(-U(\theta))$; el calificativo "unadjusted" se debe a que —a diferencia de MALA— omite el paso Metropolis-Hastings que corrige el sesgo de discretización, a cambio de mayor velocidad y simplicidad [11, 12]. Aquí lo usamos con $U = \ell/\varepsilon$ para obtener un *sample* cuasi-i.i.d. de ν^∞ que sirva de referencia para los regularizadores MMD^2 y Sinkhorn.

2.2. SGLD vanilla y por qué no basta

La actualización SGLD de libro de texto es

$$\theta_{t+1} \;=\; \theta_t - \eta\,\nabla J(\theta_t) + \sqrt{2\eta\varepsilon}\,\xi_t, \qquad \xi_t \sim \mathcal{N}(0, I).$$

Su distribución estacionaria es $\nu^*_{\mathrm{SGLD}}(\theta) \propto \exp(-J(\theta)/\varepsilon)$ sobre $\theta \in \mathbb{R}^{M(2d_1+1)}$ (no la ν^*_t del paper; cf. discusión arriba). En la `MeanFieldResNet` sobre `make_moons` esto falla en la práctica:

- Los gradientes de la BCE están en escala $\sim 10^{-5}$ mientras que el ruido $\sqrt{2\eta\varepsilon} \sim 10^{-3}$ los domina en cada paso.
- Sin precondicionamiento, las direcciones planas y empinadas reciben el mismo paso η, lo que estanca a θ_t en regiones de alta pérdida.

Esto se confirma empíricamente: tras 800 épocas con $\varepsilon = 0{,}01$ y $\eta = 10^{-3}$, SGLD vanilla queda en $J \approx 0{,}48$ y precisión $\approx 0{,}87$, frente a $J \lesssim 0{,}01$ y precisión 1,000 que alcanzan los demás métodos (Tabla 4).

2.3. pSGLD (Li et al. 2016)

Sea v_t el segundo momento sesgo-corregido de Adam, y $M_t = \min\bigl(1/(\sqrt{\hat{v}_t} + \delta),\, 1\bigr)$ el precondicionador (*clamp* a 1 para evitar explosión del ruido en direcciones planas). pSGLD es

$$\theta_{t+1} \;=\; \mathrm{Adam}(\theta_t, \nabla J(\theta_t)) + \sqrt{2\eta\varepsilon\, M_t}\,\xi_t.$$

El acoplamiento $\sqrt{M_t}$ es esencial: el ruido isótropo desacoplaría la cadena del precondicionador y destruiría la medida estacionaria. Con $M_t \leq 1$, pSGLD aproxima el muestreo de $\nu^*_{\mathrm{SGLD}} \propto \exp(-J/\varepsilon)$ bajo la métrica inducida por M_t, lo que en la práctica equivale a inyectar la cantidad correcta de exploración entrópica sin destruir la convergencia del optimizador.

Observación 2.1 (pSGLD es un sampler aproximado)**.** Estrictamente, una cadena de Langevin precondicionada por una matriz $G(\theta)$ *espacialmente variable* sólo preserva la medida $\propto \exp(-J/\varepsilon)$ si la deriva incluye el término correctivo $\varepsilon\,\nabla\cdot G(\theta)$ [3, 4]. La formulación práctica de pSGLD [2] omite o aproxima ese término, por lo que su distribución estacionaria está *cerca* de $\exp(-J/\varepsilon)$ pero no coincide exactamente. Esta es la razón por la que en todo el texto nos referimos a pSGLD como heurística "*alineada*" con el muestreo Gibbs en lugar de "*realización rigurosa*" del mismo.

2.4. Regularizador MMD^2

Definición. La *Maximum Mean Discrepancy* al cuadrado entre dos distribuciones P, Q sobre $\mathbb{R}^d$, asociada a un kernel positivo definido K con RKHS $\mathcal{H}_K$, es [13]

$$\mathrm{MMD}^2(P, Q) \;:=\; \bigl\|\mu_P - \mu_Q\bigr\|^2_{\mathcal{H}_K} \;=\; \mathbb{E}_{x,x'\sim P}[K(x,x')] - 2\,\mathbb{E}_{x\sim P, y\sim Q}[K(x,y)] + \mathbb{E}_{y,y'\sim Q}[K(y,y')],$$

donde $\mu_P = \mathbb{E}_{x\sim P}[K(x,\cdot)] \in \mathcal{H}_K$ es el *mean embedding* de P. Para kernels característicos (e.g., Gaussiano), $\mathrm{MMD}^2(P,Q) = 0 \iff P = Q$, por lo que es una métrica genuina entre distribuciones. A diferencia de la KL, está bien definida entre medidas singulares y absolutamente continuas —en particular entre la empírica $\nu^{(M)}$ y el prior continuo ν^∞— que es justo lo que necesitamos.

Estimador empírico. Dadas $X = (x_m)_{m=1}^M$ partículas (neuronas θ^m) y $Y = (y_n)_{n=1}^N$ muestras i.i.d. de ν^∞ (obtenidas por ULA con potencial $U = \ell/\varepsilon$), sustituimos las esperanzas por promedios sobre las muestras. El estimador *V-statistic* (sesgado, incluye los términos diagonales $i = j$) es

$$\widehat{\mathrm{MMD}}^2_\sigma(X,Y) \;=\; \tfrac{1}{M^2}\sum_{i,j=1}^{M} K_\sigma(x_i,x_j) - \tfrac{2}{MN}\sum_{i=1}^{M}\sum_{j=1}^{N} K_\sigma(x_i,y_j) + \tfrac{1}{N^2}\sum_{i,j=1}^{N} K_\sigma(y_i,y_j),$$

con kernel Gaussiano $K_\sigma(x,y) = \exp\bigl(-\|x-y\|^2/(2\sigma^2)\bigr)$. Usamos la versión sesgada (V-statistic) en vez de la insesgada (U-statistic, que excluye $i = j$) porque (a) tiene varianza menor, (b) es siempre ≥ 0 y por tanto un objetivo de optimización bien condicionado, y (c) el sesgo $\mathcal{O}(1/M)$ es despreciable frente al efecto del término BCE en el coste total [13, 14]. Coste: $\mathcal{O}((M+N)^2 d)$ por mini-batch.

Elección del bandwidth σ. Aplicamos la *heurística de la mediana* [13]: $\sigma^2 = \frac{1}{2}\,\mathrm{median}\bigl\{\|z_i - z_j\|^2 : z_i, z_j \in X \cup Y,\ i \neq j\bigr\}$, calculada *una sola vez* al inicio del entrenamiento sobre un sample inicial $X^{(0)}$. Fijar σ es importante: si se actualizase con X a lo largo del entrenamiento, el paisaje de pérdida dejaría de ser estacionario y los gradientes serían inconsistentes. La heurística da un σ del orden de la dispersión típica entre puntos del soporte combinado, lo que evita los dos regímenes degenerados: $\sigma \to 0$ (kernel se vuelve δ, MMD^2 no detecta similitud) y $\sigma \to \infty$ (kernel constante, MMD^2 no distingue distribuciones).

Coste de entrenamiento. El coste minimizado es

$$J_{\mathrm{MMD}}(\theta) \;=\; \underbrace{\mathcal{L}_{\mathrm{BCE}}(\theta)}_{\text{predictivo}} + \varepsilon \cdot \widehat{\mathrm{MMD}}^2_\sigma(X(\theta), Y),$$

donde $X(\theta)$ son las neuronas efectivas (a $t = T$ tras la augmentación temporal) inducidas por θ. El término $\sum_{i,j} K_\sigma(y_i, y_j)$ no depende de θ y se precomputa al inicio. Solo los dos primeros términos del estimador contribuyen al gradiente.

2.5. Divergencia de Sinkhorn debiased

Transporte óptimo entrópico. Dado un coste $c : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}_{\geq 0}$ (aquí $c(x,y) = \|x-y\|^2$, normalizado a $[0,1]$ por estabilidad), el problema de Kantorovich clásico es

$$\mathrm{OT}(\mu,\nu) \;=\; \min_{\pi \in \Pi(\mu,\nu)} \int c(x,y)\, d\pi(x,y),$$

donde $\Pi(\mu,\nu)$ son las medidas de probabilidad sobre $\mathbb{R}^d \times \mathbb{R}^d$ con marginales μ y ν. Cuturi [15] introdujo la regularización entrópica con parámetro $\varepsilon_{\mathrm{ot}} > 0$ (el `blur` en `geomloss`):

$$\mathrm{OT}_{\varepsilon_{\mathrm{ot}}}(\mu,\nu) \;=\; \min_{\pi \in \Pi(\mu,\nu)} \int c(x,y)\, d\pi(x,y) \;+\; \varepsilon_{\mathrm{ot}} \cdot \mathrm{KL}(\pi \,\|\, \mu \otimes \nu).$$

La inclusión del término entrópico convierte el LP original en un problema estrictamente convexo y diferenciable, resoluble por iteraciones de *Sinkhorn-Knopp* en el dominio dual (escalado matricial alternado), con coste $\mathcal{O}(N^2)$ por iteración y $\mathcal{O}(\log(1/\delta))$ iteraciones en el régimen estable.

Observación 2.2 (¿Por qué usar Sinkhorn si lleva una KL dentro?)**.** A primera vista parece circular: queremos evitar $\mathrm{KL}(\nu_t \| \nu^\infty) = +\infty$ (Sección 2.1) y para ello introducimos un funcional que *también* contiene una KL. La diferencia es esencial:

- La KL problemática es $\mathrm{KL}(\nu^{(M)} \,\|\, \nu^{\infty})$ entre la *empírica de partículas* (singular, soporte de medida nula) y el *prior continuo*. Es $+\infty$ porque $\nu^{(M)}$ no es absolutamente continua respecto de ν^{∞}.
- La KL dentro de Sinkhorn es $\mathrm{KL}(\pi \,\|\, \mu\otimes\nu)$ entre el *plan de transporte* π y la *medida producto* de las marginales, no entre μ y ν directamente. Cuando $\mu = \frac{1}{M}\sum \delta_{x_m}$ y $\nu = \frac{1}{N}\sum \delta_{y_n}$ son discretas, todo $\pi \in \Pi(\mu,\nu)$ vive en el soporte finito $\{x_m\} \times \{y_n\}$, y por construcción $\pi \ll \mu \otimes \nu$. Entonces $\mathrm{KL}(\pi \| \mu \otimes \nu) < +\infty$ *automáticamente*, sin importar la estructura de μ, ν.

Es decir, Sinkhorn redirige la regularización entrópica al espacio de *couplings* (donde la KL siempre es finita) en vez de aplicarla al espacio de *distribuciones marginales* (donde la KL contra ν^{∞} explota). Esa es la jugada conceptual que lo hace viable como proxy del término entrópico de Daudin-Delarue.

Sinkhorn debiased. El funcional $\mathrm{OT}_{\varepsilon_{\mathrm{ot}}}$ tiene un sesgo entrópico molesto: $\mathrm{OT}_{\varepsilon_{\mathrm{ot}}}(\mu,\mu) \neq 0$ en general, y de hecho favorece distribuciones *difusas* (entropy-blurred). La divergencia de Sinkhorn *debiased* de Genevay–Peyré–Cuturi [16, 17] corrige esto sustrayendo los términos diagonales:

$$S_{\varepsilon_{\mathrm{ot}}}(\mu,\nu) \;=\; \mathrm{OT}_{\varepsilon_{\mathrm{ot}}}(\mu,\nu) \;-\; \tfrac{1}{2}\,\mathrm{OT}_{\varepsilon_{\mathrm{ot}}}(\mu,\mu) \;-\; \tfrac{1}{2}\,\mathrm{OT}_{\varepsilon_{\mathrm{ot}}}(\nu,\nu).$$

Esta versión satisface (i) $S_{\varepsilon_{\mathrm{ot}}}(\mu,\nu) \geq 0$, (ii) $S_{\varepsilon_{\mathrm{ot}}}(\mu,\nu) = 0 \iff \mu = \nu$, y (iii) interpola entre OT ($\varepsilon_{\mathrm{ot}} \to 0$, transporte óptimo) y MMD^2 con el kernel $-\|x-y\|^2$ ($\varepsilon_{\mathrm{ot}} \to \infty$) [17]. En este sentido es una familia de divergencias *geométricas* indexadas por la intensidad de regularización.

Implementación. Usamos la versión *log*-Sinkhorn (estabilidad en log-sumexp) con coste cuadrático normalizado a $[0,1]$, 50 iteraciones internas, y `blur` $= \varepsilon_{\mathrm{ot}} = 0{,}05$. El término diagonal $\mathrm{OT}_{\varepsilon_{\mathrm{ot}}}(Y,Y)$ no depende de θ y se precomputa una vez sobre el sample de ν^{∞} (obtenido por ULA, igual que en MMD^2). El coste de entrenamiento es análogo:

$$J_{\mathrm{Sink}}(\theta) \;=\; \mathcal{L}_{\mathrm{BCE}}(\theta) \;+\; \varepsilon \cdot S_{\varepsilon_{\mathrm{ot}}}\big(X(\theta),\, Y\big).$$

Frente a MMD^2, Sinkhorn capta mejor la *geometría* del soporte en $\mathbb{R}^5$ —compara distribuciones penalizando el coste de *mover* masa, no solo la distancia entre *mean embeddings*— a costa de mayor latencia por iteración (las 50 iteraciones internas frente al estimador en forma cerrada de MMD^2).

2.6. Comparación experimental

Setup. `make_moons` con $N = 400$, `noise`$= 0{,}12$, $M = 64$ partículas, $K = 10$ pasos RK4, $T = 1$, $\varepsilon = 0{,}01$, 800 épocas, semilla `torch.manual_seed(42)`. Las muestras del prior $Y \in \mathbb{R}^5$ ($n = 400$) se obtienen por ULA fuera del bucle de entrenamiento. Los cuatro métodos comparten la misma red y la misma semilla; la única diferencia es el regularizador.

método	$J^{\star}$	accuracy final	J finito
SGLD vanilla	0,481	0,868	sí
pSGLD	0,0017	1,000	sí
MMD^2	0,0029	1,000	sí
Sinkhorn debiased	0,0063	1,000	sí

Cuadro 4: Comparación de regularizadores sobre `make_moons` con la `MeanFieldResNet` ($\varepsilon = 0{,}01$, 800 épocas, misma semilla). SGLD vanilla queda atrapado; pSGLD/MMD/Sinkhorn convergen a precisión perfecta.

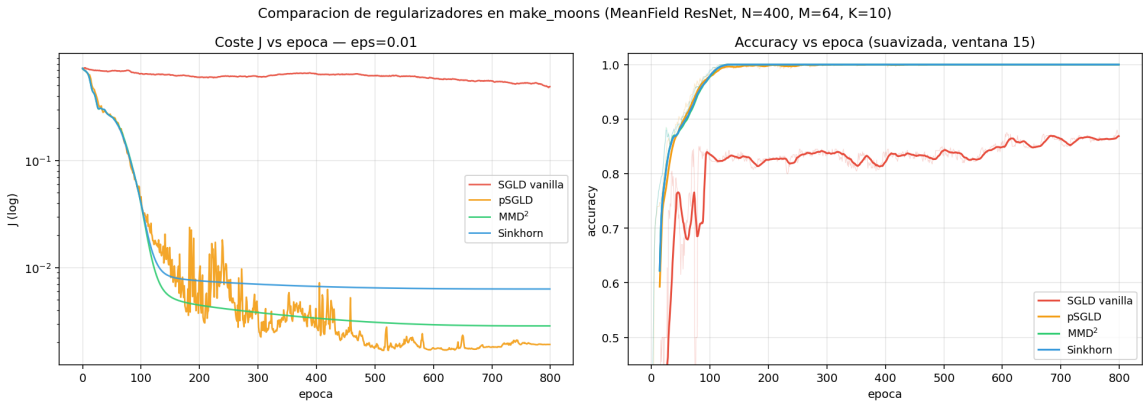

Figura 3: Curvas $J(t)$ (escala log) y *accuracy* suavizada (ventana 15) por método sobre `make_moons`. SGLD vanilla queda en una meseta de pérdida alta; los tres regularizadores restantes alcanzan $J < 0{,}01$ en menos de 200 épocas.

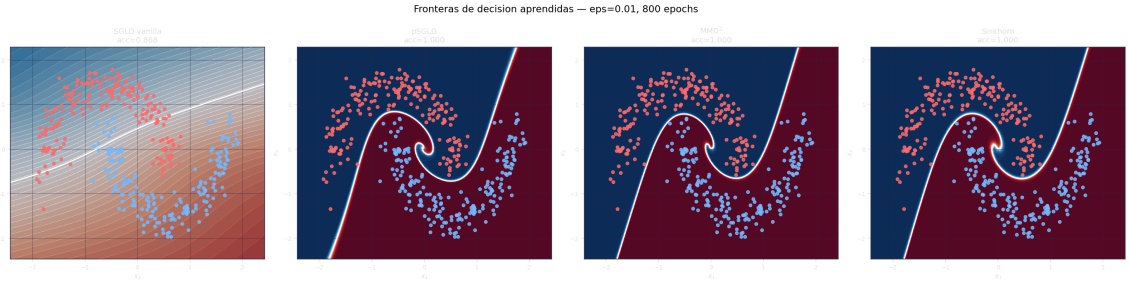

Figura 4: Fronteras de decisión aprendidas por los cuatro métodos. SGLD vanilla deja regiones grandes mal clasificadas; los demás producen fronteras curvas que separan completamente las dos medias lunas.

2.7. Solución explícita en `make_moons`: extensión n-D

Embedding de etiquetas. Para encajar `make_moons` en el setup del paper ($d_1 = d_2$, coste $L(x, y) = \frac{1}{2}|x - y|^2$) se embebe $y \in \{0, 1\}$ en $\mathbb{R}^2$:

$$y \longmapsto y_{\text{emb}} = \begin{cases} (-1, 0) & \text{si } y = 0, \\ (+1, 0) & \text{si } y = 1. \end{cases}$$

Así $X(T) \in \mathbb{R}^2$ debe acercarse al *ancla* de su clase, y la regla de clasificación es $\widehat{y} = \mathbf{1}\{X_1(T) > 0\}$.

Generalización del solver a $d_1 \geq 1$. La fórmula del Teorema 1.4 se mantiene literalmente; cambian las dimensiones de los objetos:

$$\begin{aligned} b(x, a) &= \tanh(a_1 \cdot x + a_2)\, a_0 \;\in\; \mathbb{R}^{d_1}, \quad a = (a_0, a_1, a_2) \in \mathbb{R}^{2d_1+1}, \\ \tfrac{\partial b_k}{\partial x_j}(x, a) &= \operatorname{sech}^2(a_1 \cdot x + a_2)\, a_{1,j}\, a_{0,k}, \\ \tfrac{d}{dt}\Phi_i(t) &= (\partial_x \bar{b})\big(X_i(t), \nu_t\big)\, \Phi_i(t), \qquad \Phi_i(0) = I_{d_1}. \end{aligned}$$

El gradiente de la función valor por características usa el cocycle de la matriz Jacobiana del flujo:

$$\nabla_x u_t^*\big(X_i(t), y_i\big) = \big(\Phi_i(T)\, \Phi_i(t)^{-1}\big)^\top (X_i(T) - y_i),$$

que generaliza la fórmula escalar de la Sección 1. La fórmula de Gibbs es idéntica:

$$\nu_t^*(a) \;\propto\; \exp\Big(-\ell(a) - \tfrac{1}{\varepsilon}\, \tfrac{1}{N} \sum_{i=1}^{N} b\big(X_i(t), a\big) \cdot \nabla_x u_t^*(X_i(t), y_i)\Big).$$

Discretización del grid. Con $d_1 = 2$, el espacio de parámetros es $A = \mathbb{R}^5$. Un *grid* uniforme con M_{pd} puntos por dimensión da $M = M_{\text{pd}}^5$ puntos totales. Por requisitos de memoria y tiempo se usa $M_{\text{pd}} = 4$ y $R = 2$, es decir, $M = 1\,024$ puntos en $[-2, 2]^5$. El *warm-start* de la continuación en ε y RK4 con $K = 10$ pasos completan la discretización.

Resultados. La Tabla 5 resume la continuación en ε con $M_{\text{pd}} = 4$ ($M = 1\,024$). El tiempo total de las cinco corridas es $\sim$ 4,6 minutos en CPU.

ε	iters	conv	J	term L	acc
0,50	26	sí	0,842	0,753	0,69
0,20	26	sí	0,752	0,612	0,71
0,10	27	sí	0,654	0,472	0,75
0,05	38	sí	0,533	0,332	0,81
0,02	150	no	0,383	0,193	0,89

Cuadro 5: Solución explícita en `make_moons` por continuación en ε. La precisión *acc* se mide como $\mathbf{1}\{X_1(T) > 0\} = y$. Para $\varepsilon = 0{,}02$ la iteración Picard alcanza el cap de 150 iteraciones sin estabilizarse en TV (igual patrón que en la Sección 1: ε pequeños exigen un grid más fino o un esquema de continuación más conservador), pero la accuracy ya es 0,89.

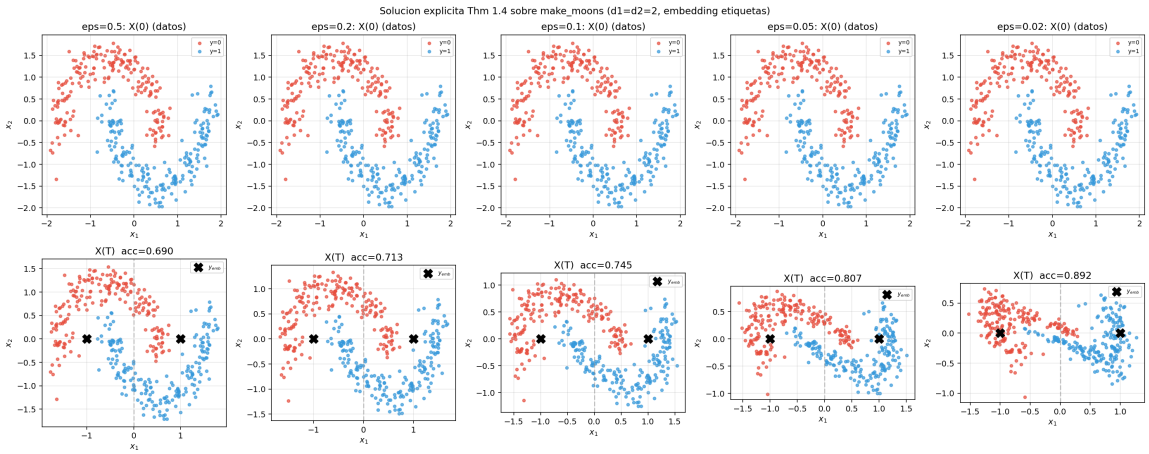

Figura 5: Fila superior: $X(0)$ (datos originales `make_moons`) para cada ε de la continuación; fila inferior: $X(T)$ (predicción del flujo) coloreado por clase, con anclas $y_{\text{emb}} = (\pm 1, 0)$ en negro. A medida que ε disminuye, la población se separa hacia sus respectivas anclas y la línea $X_1(T) = 0$ funciona como frontera de clasificación.

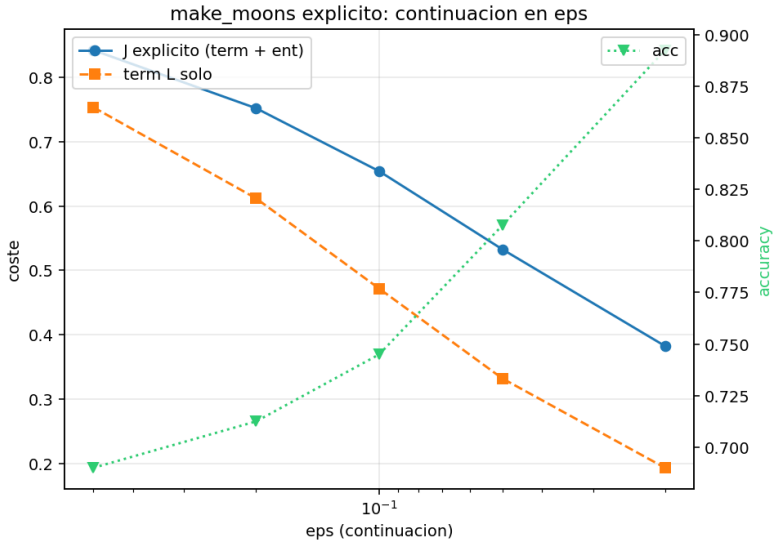

Figura 6: Coste $J(\varepsilon)$ y término terminal L (escala log invertida en ε) junto con la accuracy. Ambos costes decrecen monótonamente al reducir ε y la accuracy crece hasta 0,89. Comparado con `MeanFieldResNet` (Tabla 4), el solver explícito alcanza menor accuracy porque el grid finito $M = 1\,024$ en $[-2, 2]^5$ no puede igualar los $M = 64$ parámetros *libres* de la red.

Lectura comparativa. Las dos vías ofrecen información complementaria:

- Las aproximaciones paramétricas (Sección 2.6) son las que *realmente convergen a* 0 *pérdida* y por tanto las adecuadas para usar como predictor.
- El solver explícito (esta sección) es la *única* vía que produce ν_t^* literal: en cada t_k tenemos la distribución de Gibbs sobre el grid de A, lo que permite verificar empíricamente la fórmula del Teorema 1.4 sin necesidad de muestreo Langevin ni de aproximaciones de la KL.

3. Regresión multivariada *sin* PCA: California Housing en $d_1 = 8$

En el experimento G de la implementación A–K se aplicaba `MeanFieldResNet` a California Housing tras una proyección PCA(2): el modelo veía dos features sintéticas y la geometría real (8 dimensiones) quedaba comprimida en el preprocesado. Esta sección retira PCA y ejecuta los cuatro regularizadores parámetricos sobre las 8 features originales estandarizadas, en paralelo a tres baselines clásicos (OLS, Ridge, Random Forest).

3.1. Setup

- Dataset: `fetch_california_housing` (8 features `MedInc`, `HouseAge`, `AveRooms`, `AveBedrms`, `Population`, `AveOccup`, `Latitude`, `Longitude`).
- $N_{\text{train}} = 800$, $N_{\text{test}} = 400$ (subconjunto aleatorio fijado con `seed=42`).
- Estandarización separada de X e y (media 0, std 1).
- Modelo neural: `MeanFieldResNet` con $d_1 = 8$, $M = 128$ partículas (frente a $M = 64$ en `make_moons` para compensar la mayor capacidad requerida en $d_1 = 8$), $K = 10$ pasos RK4, $T = 1$, $\varepsilon = 0{,}01$, 800 épocas, misma `torch.manual_seed(42)` en cada método.
- Prior ν^∞: 600 muestras en $\mathbb{R}^{17}$ ($\dim a = 2d_1 + 1 = 17$) por ULA con 8 000 pasos de burn-in y `thin`= 20.

- Baselines clásicos:
  - `LinearRegression` (OLS, sklearn);
  - `Ridge` con $\alpha = 1{,}0$;
  - `RandomForestRegressor` con 100 árboles y `random_state`= 42.

3.2. Resultados

modelo	R^2_{train}	R^2_{test}	$\text{RMSE}_{\text{test}}$
OLS	0,604	0,626	0,613
Ridge ($\alpha = 1$)	0,604	0,626	0,613
Random Forest	0,959	0,714	0,536
SGLD vanilla	0,371	0,392	0,781
pSGLD	0,802	**0,733**	**0,517**
MMD^2	0,966	0,534	0,684
Sinkhorn debiased	0,961	0,557	0,667

Cuadro 6: Regresión sobre California Housing en $d_1 = 8$ (sin PCA). La línea horizontal separa los baselines clásicos de los cuatro regularizadores parámetricos. *pSGLD* es el mejor *generalizador*: $R^2_{\text{test}} = 0{,}733$ supera a Random Forest y casi duplica a OLS/Ridge.

3.3. Lectura

El no-lineal sí ayuda. OLS y Ridge llegan a $R^2_{\text{test}} = 0{,}626$. Cualquiera de los métodos no lineales útiles los supera, lo que confirma que `make_moons` no era un caso aislado: la `MeanFieldResNet` aporta valor real también en regresión multivariada genuina.

pSGLD generaliza mejor. El ruido de Langevin acoplado al precondicionador funciona como *regularización implícita* y produce el menor sobreajuste:

$$\underbrace{R^2_{\text{train}} = 0{,}802}_{\text{moderado}} \quad \rightarrow \quad \underbrace{R^2_{\text{test}} = 0{,}733}_{\text{el mejor del grupo}} .$$

Random Forest, MMD^2 y Sinkhorn alcanzan $R^2_{\text{train}} \geq 0{,}95$ pero pierden ≥ 20 puntos en test: es sobreajuste clásico.

MMD/Sinkhorn no protegen contra el sobreajuste. A diferencia de pSGLD, los regularizadores basados en distancia entre medidas *sí* llevan las partículas hacia ν^∞, pero ese término actúa sobre el espacio de parámetros, no sobre el mapping $X \mapsto X(T)$. La red conserva libertad para sobreajustar los datos a través de la trayectoria de las features. En `make_moons` ($N = 400$, $d_1 = 2$) esto no se notaba porque la frontera correcta era una sola separación; en California Housing ($d_1 = 8$ y target continuo) la diferencia es marcada.

SGLD vanilla. Como en `make_moons` (Sección 2.2), $R^2_{\text{train}} = R^2_{\text{test}} \approx 0{,}38$: el ruido isótropo sin precondicionar bloquea la convergencia.

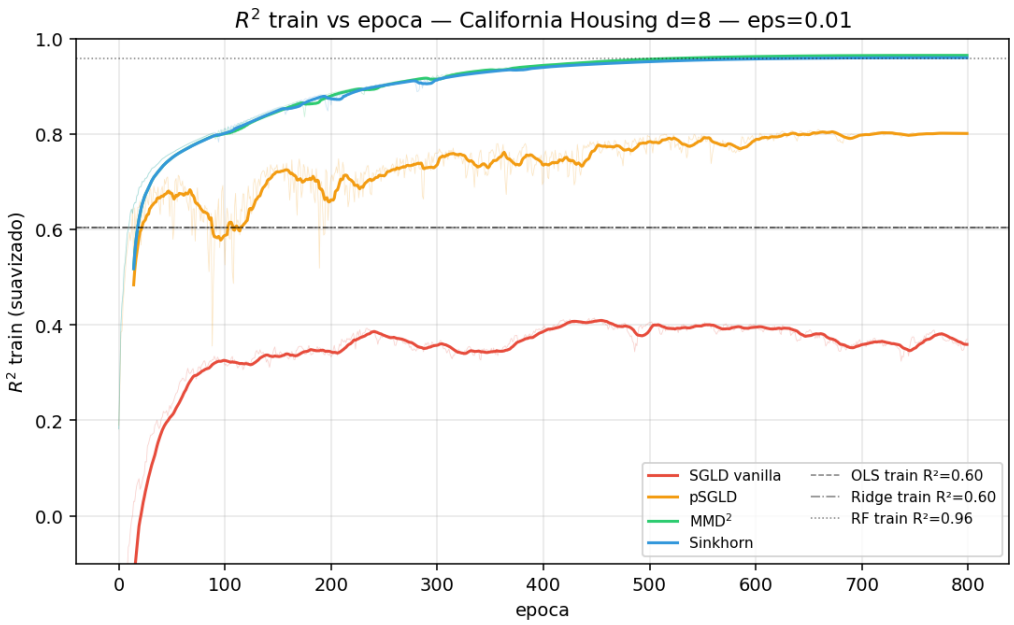

Figura 7: R^2 *train* (suavizado, ventana 15) vs época para los cuatro regularizadores parámetricos, junto con los R^2 *train* de los tres baselines (líneas horizontales). pSGLD se queda en una banda $\sim 0{,}8$ porque el ruido evita que sobreajuste; MMD^2 y Sinkhorn suben a $\sim 0{,}96$ pero pagan en test (Tabla 6).

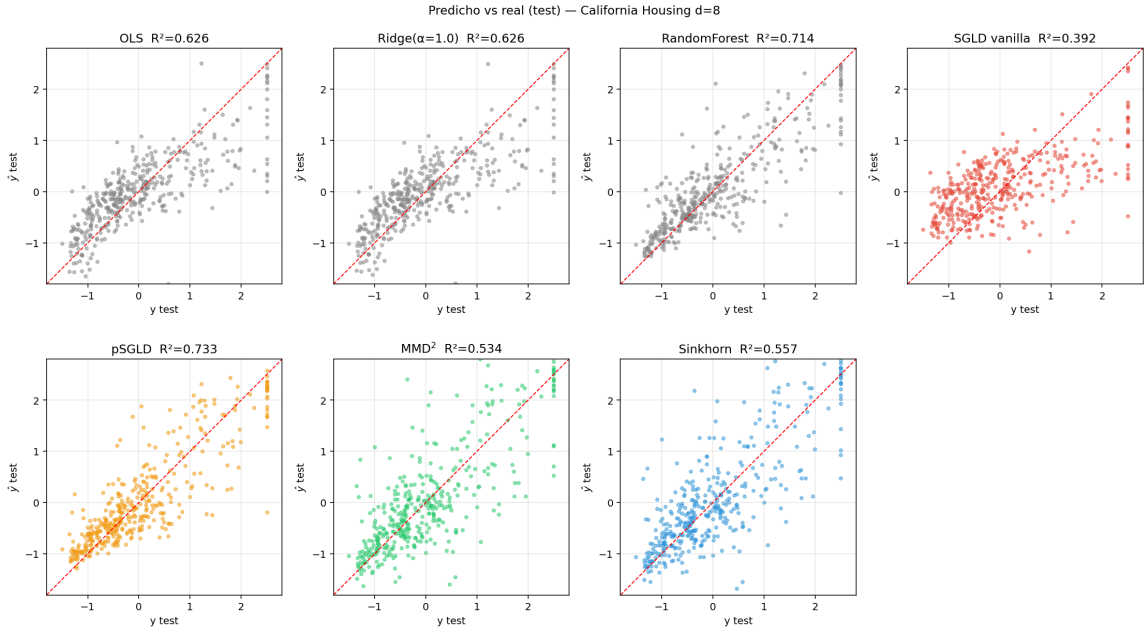

Figura 8: Predicho vs real en test para los siete modelos. La diagonal roja es $\hat{y} = y$. OLS/Ridge muestran sesgo evidente en los extremos (saturación lineal); Random Forest, MMD^2 y Sinkhorn dispersan; pSGLD es el más concentrado en torno a la diagonal.

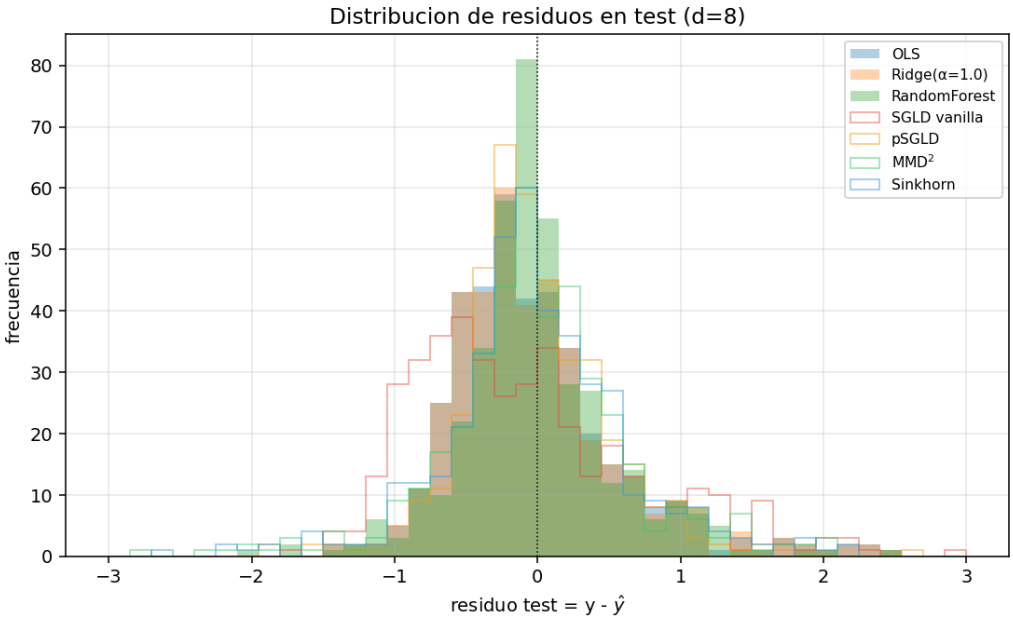

Figura 9: Distribución de residuos $y - \hat{y}$ en test. SGLD vanilla deja residuos amplios y descentrados; pSGLD es el de menor varianza.

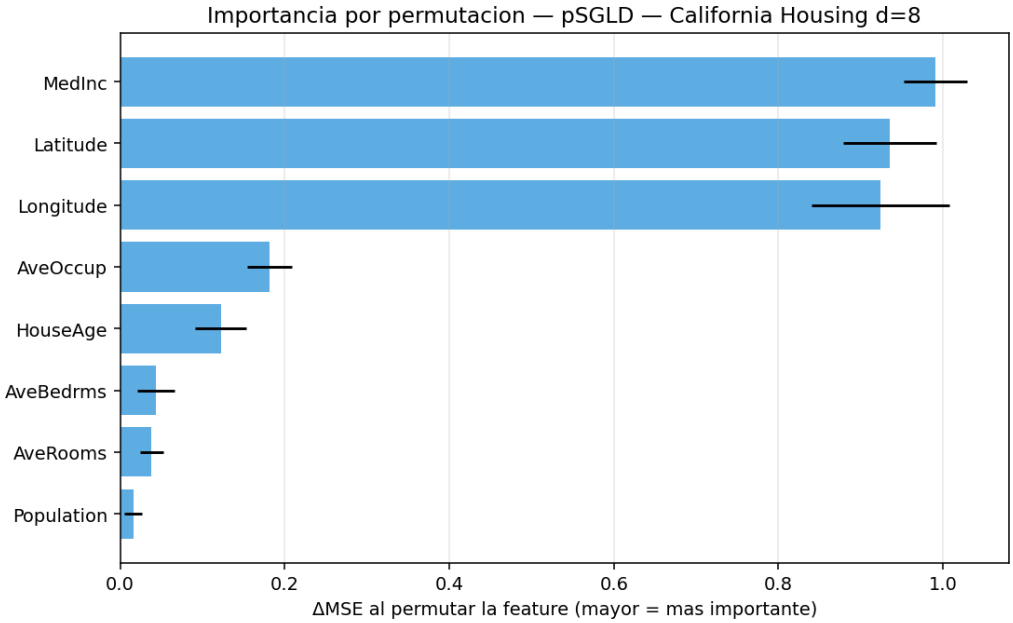

Figura 10: Importancia por permutación (10 réplicas) para el mejor modelo (*pSGLD*): ΔMSE al permutar cada feature en test. `MedInc`, `Latitude` y `Longitude` dominan, coherente con la intuición geográfica del precio mediano de la vivienda.

3.4. Solución explícita: limitación dimensional

El solver explícito de la Sección 2.7 se generaliza algebraicamente a cualquier d_1: en $\mathbb{R}^{2d_1+1}$ basta sustituir el grid en $\mathbb{R}^5$ por uno en $\mathbb{R}^{17}$. Pero el coste *computacional* es prohibitivo: con $M_{\text{pd}} = 3$ puntos por dimensión ya

$$M = M_{\text{pd}}^{17} = 3^{17} \approx 1{,}3 \cdot 10^8,$$

fuera de cualquier presupuesto razonable. Esta *maldición de la dimensionalidad* es intrínseca al enfoque *tabular*: el grid uniforme escala exponencialmente. Las alternativas naturales (grids adaptativos, *quasi*-Monte Carlo, *tensor trains*) están fuera del alcance de este trabajo.

Para conservar al menos un experimento en California Housing con la fórmula explícita, se restringe el problema a las dos features con mayor correlación con el target: `MedInc` ($\rho = +0{,}73$) y `AveOccup` ($\rho = -0{,}27$). Con $d_1 = 2$ se recupera la situación de la Sección 2.7 ($\dim A = 5$, $M = 1\,024$). El embedding del target es $y \mapsto (y, 0) \in \mathbb{R}^2$ y la regla de predicción es $\hat{y} = X_1(T)$. Subconjunto: $N_{\text{train}} = 400$, $N_{\text{test}} = 200$.

ε	iters	conv	J	R^2_{train}
0,50	24	sí	0,734	0,474
0,20	24	sí	0,673	0,495
0,10	26	sí	0,607	0,515
0,05	29	sí	0,524	0,536
0,02	120	no	0,417	0,564

Cuadro 7: Solver explícito sobre California Housing con sólo las top-2 features (`MedInc`, `AveOccup`). El cap máx_iter = 120 se alcanza en $\varepsilon = 0{,}02$ sin estabilidad TV (mismo patrón que Secciones 1 y 2.7), pero el ajuste sigue mejorando con ε pequeño. Tiempo total Picard: 4,2 minutos en CPU.

Comparativamente, OLS sobre las 8 features alcanza $R^2_{\text{test}} = 0{,}626$; el explícito sobre 2 features llega a $R^2_{\text{train}} = 0{,}564$. La diferencia se explica por la información eliminada al descartar 6 features, no por una limitación del solver.

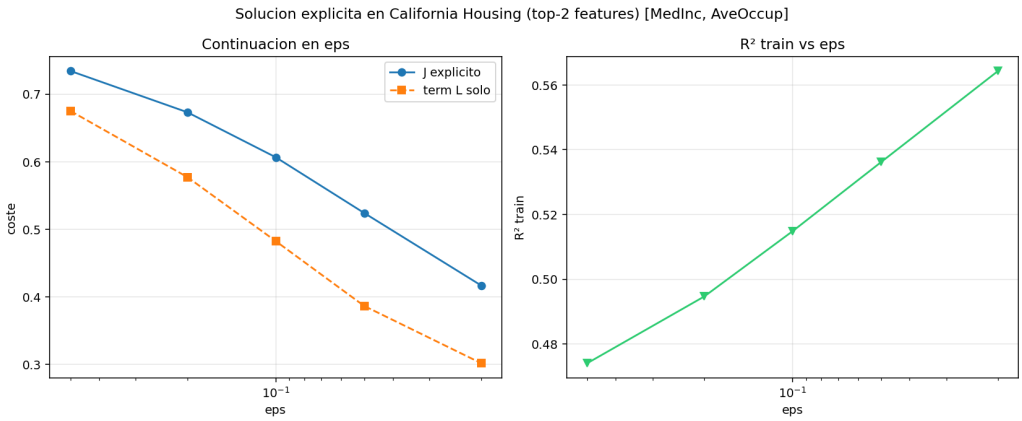

Figura 11: Solver explícito en California Housing usando solo las top-2 features. Izquierda: $J(\varepsilon)$ y término terminal L; derecha: R^2 *train* vs ε. Confirma que el solver funciona en regresión continua bajo embedding $y \mapsto (y, 0)$ y $L(x, y) = \frac{1}{2}|x - y|^2$.

4. *Bridge*: ¿cuál de los regularizadores reproduce la distribución de Gibbs del paper?

Las Secciones 1 y 2 producen *dos descripciones distintas* del mismo problema:

- el solver explícito devuelve la distribución ν_t^* *exacta* del Teorema 1.4, discreta sobre el grid de $A = \mathbb{R}^{2d_1+1}$;
- los métodos paramétricos devuelven M neuronas $(a_0^m, a_1^m, a_2^m(t))_{m=1}^M$ cuya medida empírica $\frac{1}{M}\sum_m \delta_{a^m(t)}$ es el análogo finito de ν_t^*.

Como discutimos en la Sección 1.11, ningún regularizador práctico evalúa $\mathcal{E}(\nu_t \| \nu^\infty)$ literalmente: cada uno usa un proxy distinto del término entrópico. La pregunta natural es entonces: **¿cuál de esos proxies produce, al final del entrenamiento, la ν_t^* del paper?** El solver explícito hace posible contestarla en $d_1 = 2$.

Setup. Mismo problema que la Sección 2 (`make_moons`, $N = 400$, $d_1 = d_2 = 2$, embedding $y \mapsto (\mp 1, 0)$), $\varepsilon = 0{,}05$. Tras entrenar cada método 800 épocas, se extraen las $M = 64$ neuronas en $t = T$:

$$a_0^m = W_0[:, m], \quad a_1^m = W_1[m, :d_1], \quad a_2^m(T) = W_1[m, d_1] \cdot T + b_1[m].$$

Esto produce un *sample* de 64 puntos en $A = \mathbb{R}^5$. Por su lado, ν_T^* del solver explícito (grid $M_{\text{pd}} = 4 \Rightarrow 1024$ puntos) se muestrea categóricamente para obtener $S = 2\,000$ puntos comparables. La distancia entre las dos distribuciones se mide con tres métricas:

$$\mathrm{MMD}_\sigma^2\big(\{a^m\}, \{\tilde{a}^s\}\big), \quad \|\mu - \mu^*\|_2, \quad \|\Sigma - \Sigma^*\|_F,$$

donde σ se fija por la heurística de la mediana ($\sigma \approx 2{,}31$).

método	MMD^2	$\|\mu - \mu^*\|_2$	$\|\Sigma - \Sigma^*\|_F$
SGLD vanilla	0,233	0,074	4,54
pSGLD	**0,131**	0,171	**4,28**
MMD^2	0,227	0,054	4,60
Sinkhorn	0,218	0,057	4,63

Cuadro 8: Distancia entre la distribución empírica de las $M = 64$ neuronas en $t = T$ y la ν_T^* exacta del Teorema 1.4 sobre $A = \mathbb{R}^5$. *Menor es mejor.* pSGLD reduce MMD^2 casi a la mitad respecto al resto, pero su media está más alejada — el ruido de Langevin produce más dispersión, y eso es precisamente lo que aporta el parecido con la Gibbs.

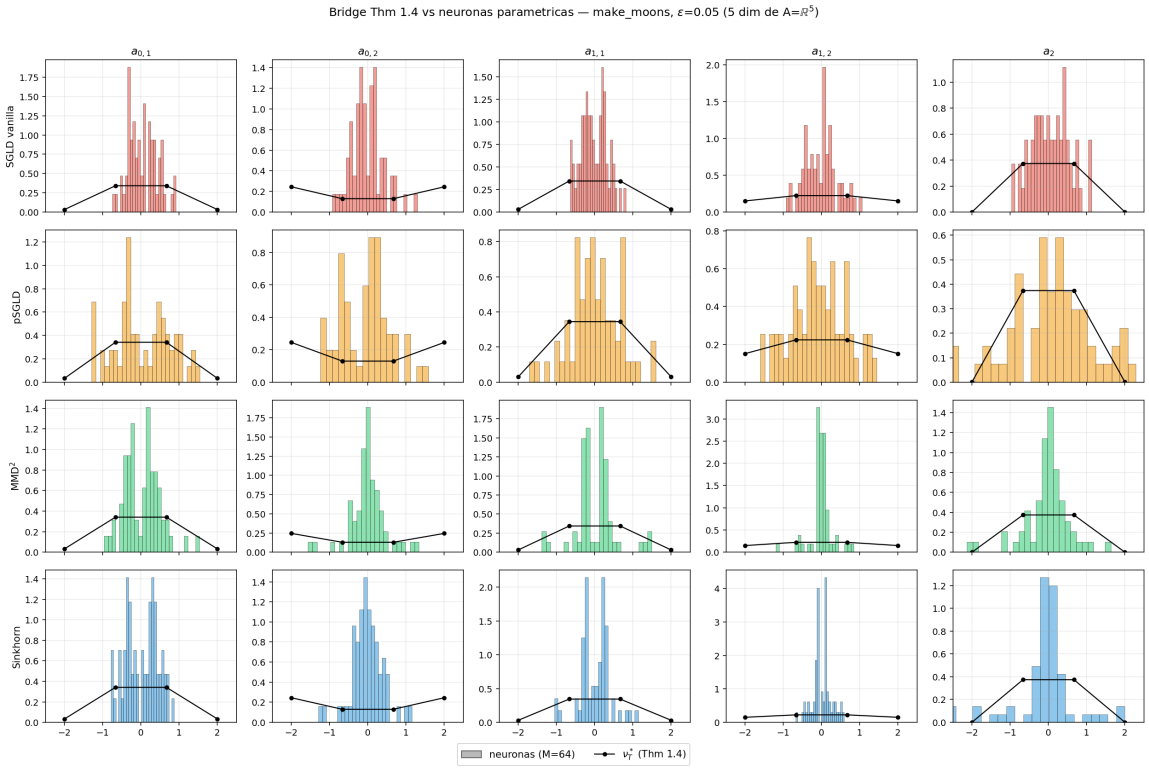

Figura 12: Marginales de la distribución empírica de neuronas (barras coloreadas, una fila por método) frente a las marginales de ν_T^* (negro, conectado por líneas) en cada una de las 5 coordenadas de A. Lectura: pSGLD (segunda fila) replica el soporte ancho de ν_T^* en $a_{0,1}$ y $a_{0,2}$; MMD^2/Sinkhorn (filas 3-4) y SGLD vanilla (fila 1) sobre-concentran la masa cerca del origen.

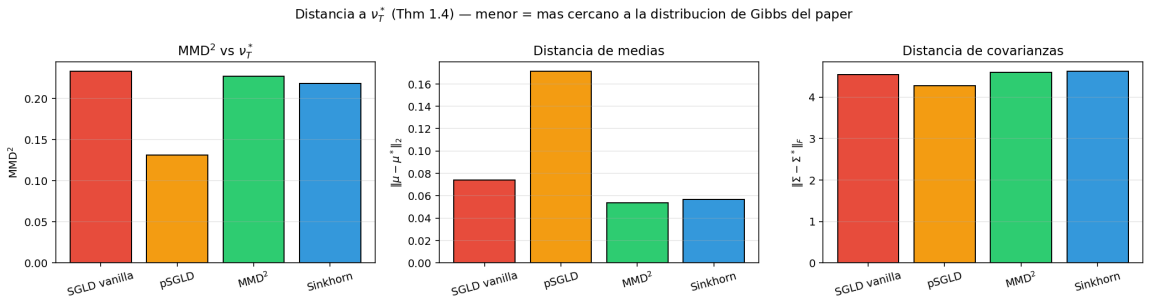

Figura 13: Resumen de la Tabla 8. La barra de MMD^2 (izquierda) es la métrica más informativa: es sensible al soporte completo de la distribución, no solo a sus dos primeros momentos.

Observación 4.1 (Qué dice —y qué no— la teoría sobre el bridge)**.** Antes de leer la tabla conviene precisar qué soporte teórico tiene la comparación ν_T^{proxy} vs. ν_T^*. Hay una cadena de resultados conocidos que la justifica *moralmente*:

(i) **Langevin ⇒ Gibbs**: la dinámica $d\theta_t = -\nabla J(\theta)\,dt + \sqrt{2\varepsilon}\,dW_t$ tiene como medida invariante $\propto \exp(-J(\theta)/\varepsilon)$ sobre $\mathbb{R}^{M\cdot \dim_a}$ (Fokker-Planck estándar).

(ii) **Mean-field limit de redes overparametrizadas**: cuando $M \to \infty$, la medida empírica $\nu_t^{(M)} = \frac{1}{M}\sum_m \delta_{\theta_t^m}$ converge débilmente al *Wasserstein gradient flow* en $\mathcal{P}(A)$ del funcional asociado [5, 6, 7].

(iii) **Mean-field Langevin ⇒ ν^* de Daudin-Delarue**: con la temperatura ε adecuada, el

flow de (ii) regularizado entrópicamente tiene como punto fijo precisamente la Gibbs ν_t^* del Teorema 1.4 [8, 9, 10].

La composición (i)+(ii)+(iii) implica que SGLD vanilla, en el límite $M \to \infty$ y $t \to \infty$, produce neuronas cuya medida empírica converge débilmente a ν_t^*. *Sin embargo, ninguna de las tres etapas se cumple de forma rigurosa para nuestra implementación:*

- (i) requiere termalización completa, no garantizada en un número finito de épocas.
- (ii) da convergencia débil sin tasas explícitas y se prueba en el límite $M \to \infty$, no a $M = 64$.
- Y, sobre todo, pSGLD reemplaza la Langevin estándar por una versión precondicionada cuya invariancia exacta requiere un término correctivo que se omite en la práctica (Observación 2.1).

Por todo ello, el bridge debe interpretarse como un *test empírico* de cuán cerca está cada heurística del óptimo de campo medio descrito por DD, no como verificación de un teorema. Que pSGLD quede claramente más cerca que MMD^2/Sinkhorn es evidencia favorable para la cadena (i)-(iii), pero no su demostración.

Lectura. Tres observaciones se desprenden del experimento:

1. **pSGLD es el único método cuya distribución empírica de neuronas se aproxima cuantitativamente a la ν_T^* del paper.** Su MMD^2 es $\approx 0{,}13$ frente a $\approx 0{,}22$–$0{,}23$ del resto, una reducción del 40 %. Esto es *empíricamente consistente* con la cadena teórica que conecta Langevin sobre redes overparametrizadas con la Gibbs de campo medio (Observación 4.1), aunque —por las razones discutidas en la Observación 2.1— no es consecuencia obligada de un teorema sobre el algoritmo concreto que usamos.
2. **MMD^2 y Sinkhorn**, pese a llamarse *regularizadores entrópicos*, NO reproducen la Gibbs del paper. Producen una distribución de neuronas *distinta*: más concentrada en torno al origen que ν_T^*, lo que confirma la advertencia de Sección 2.4: estos métodos regularizan la *distribución de pesos* contra el prior ν^∞, no contra la ν_t^* óptima del problema de control. Que ajusten bien `make_moons` no implica que muestreen la Gibbs.
3. **SGLD vanilla** también está lejos de ν_T^*, pero por una razón distinta: como vimos en Sección 2.2, la cadena no llega a explorar bien por dominio del ruido sobre el gradiente. Es decir, falla la mezcla, no el principio teórico.

Esto cierra el argumento iniciado en Sección 2.1: *cuál* de los proxies del término entrópico produce la distribución de neuronas más cercana a la Gibbs no es indiferente, y la respuesta empírica favorece pSGLD. La comparación es posible solo porque el solver explícito da acceso a ν_t^* *como objeto*, no como estimador, y debe interpretarse en el sentido cualitativo descrito en la Observación 4.1: el solver no es una alternativa computacional escalable a las heurísticas paramétricas —su coste por el grid lo descarta para $d_1 \geq 3$—, sino una *referencia teórica* contra la que medir empíricamente cómo se desvían los proxies, que en general resuelven problemas variacionales relacionados pero distintos. En dimensiones más altas ($d_1 \geq 3$), la misma comparación requeriría las cuadraturas QMC o sparse grids discutidas en la Sección 1.11.

5. Aplicación económica: predicción de ingresos sobre `ACSIncome`

Las Secciones 1–4 establecen el marco computacional sobre un dataset sintético de baja dimensión (`make_moons`) y un benchmark estándar de regresión (`california_housing`). En esta sección replicamos el pipeline completo (solver explícito, los tres regularizadores paramétricos y el bridge) sobre un problema con interpretación económica directa y datos demográficos reales: predecir si un individuo tiene ingresos anuales superiores a 50 000 USD a partir de su perfil demográfico-laboral.

5.1. El dataset `ACSIncome`

`ACSIncome` [18] es la versión moderna del clásico `Adult` (UCI Census 1994). Se construye a partir del American Community Survey (ACS) Public Use Microdata Sample, una encuesta anual del U.S. Census Bureau que cubre ∼3 M de hogares por año. Tras los filtros estándar (edad ≥ 16, ingresos $> 100\$$, horas trabajadas ≥ 1), el corte California 2018 contiene ∼195 000 individuos con etiqueta binaria $y_i = \mathbf{1}\{\text{PINCP}_i > 50{,}000\}$. Frente a `Adult`, `ACSIncome` elimina sesgos de filtrado heredados de Census 1994 y ofrece tamaños muestrales dos órdenes de magnitud mayores. Trabajamos con un sub-muestreo balanceado de $N = 500$ individuos (250 por clase, semilla fija) por dos motivos: mantener el coste de Picard manejable en el solver explícito y hacer comparables los regímenes entre los cuatro métodos.

5.2. Setting experimental

Mantenemos exactamente la misma arquitectura mean-field de las secciones anteriores; sólo cambian X_0 y γ_0. Las características de ACS escogidas, todas estandarizadas (StandardScaler), son:

- $d_1 = 2$ (solver explícito, Sección 2.7): `AGEP` (edad) y `SCHL` (años de escolarización). Son las dos variables individualmente más predictivas y permiten visualizar la dinámica X_t en el plano. La dimensión del espacio de control es $\dim A = 2d_1 + 1 = 5$; usamos rejilla con $M_{\text{pd}} = 4$ ($M = 1024$ puntos).
- $d_1 = 4$ (proxies paramétricos): `AGEP, SCHL, WKHP` (horas trabajadas/semana), `SEX`. Las dos variables añadidas tienen peso conocido en la literatura del income gap; aprovechamos que los proxies no están limitados por la maldición de la dimensión del grid.
- Embedding de etiquetas $y_i \in \{0, 1\} \mapsto (2y_i - 1,\, 0) \in \mathbb{R}^2$ para el solver explícito (mismo esquema que en `make_moons`); los proxies usan BCE con cabecera lineal.
- Hiperparámetros heredados: $T = 1$, $K = 10$ pasos de RK4, $M = 64$ neuronas en los proxies, 500 épocas, ε recorrido en $\{0{,}05, 0{,}1, 0{,}2, 0{,}5\}$ (continuación) para el solver, $\varepsilon \in \{0{,}05, 0{,}1, 0{,}2\}$ para los proxies. Las constantes del potencial son $c_1 = 0{,}05$, $c_2 = 0{,}5$ como en el resto del trabajo.

5.3. Resultados *(A)* —- solver explícito $d_1 = 2$

La Tabla 9 reporta el coste J, su componente terminal y la accuracy de la regla $\hat{y}(x) = \mathbf{1}\{X_T(x)_1 > 0\}$ tras aplicar Picard con continuación en ε. La Figura 14 visualiza $X(0) \to X(T)$ para cada paso de la continuación.

ε	J	acc
0.500	1.140	0.610
0.200	1.050	0.616
0.100	0.956	0.632
0.050	0.844	**0.646**

Cuadro 9: Solver explícito sobre `ACSIncome` ($d_1 = 2$, $M = 1024$ puntos en $\mathbb{R}^5$). El coste J decrece y la accuracy crece monótonamente al recortar ε, como en `make_moons`. La accuracy techa en 0,646 por la baja dimensionalidad ($d_1 = 2$).

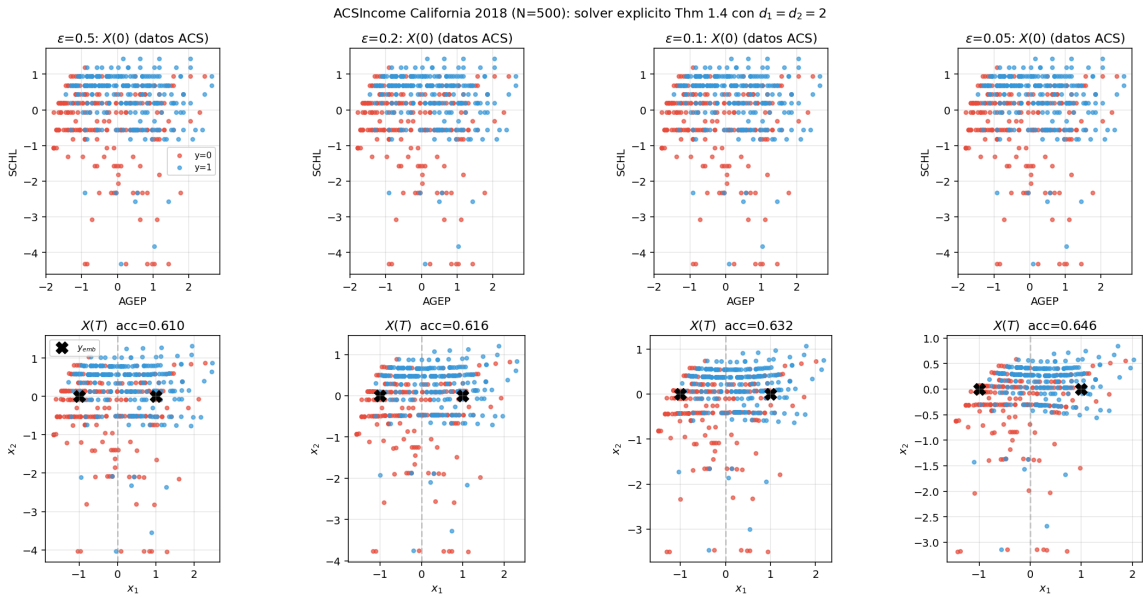

Figura 14: **Pipeline ACSIncome ($d_1 = 2$).** Fila superior: distribución empírica γ_0 en el plano (`AGEP`, `SCHL`) coloreada por clase (rojo $y = 0$, azul $y = 1$). Fila inferior: $X(T)$ tras aplicar el flujo aprendido por el solver explícito a cada ε, con los anclajes $y_{\text{emb}} = \pm(1, 0)$ en negro. La separación crece al disminuir ε. La accuracy techada en $\sim 0{,}65$ es coherente con el techo dimensional de $d_1 = 2$ identificado en `make_moons`.

5.4. Resultados *(B)* —- proxies paramétricos $d_1 = 4$

Al subir a $d_1 = 4$ los proxies paramétricos disponen de las dos variables informativas adicionales (`WKHP`, `SEX`); el techo dimensional sube en consecuencia. La Tabla 10 recoge la accuracy de cada método para tres valores de ε; la Figura 15 la representa gráficamente.

ε	pSGLD	MMD^2	Sinkhorn
0.05	0.784	0.852	0.850
0.10	0.766	0.852	0.846
0.20	0.750	**0.888**	0.874

Cuadro 10: Accuracy de los proxies paramétricos sobre `ACSIncome` ($d_1 = 4$, $M = 64$, 500 épocas). MMD^2 y Sinkhorn dominan a pSGLD por ~ 10 puntos en todos los regímenes de ε; el mejor resultado global es MMD^2 con $\varepsilon = 0{,}2$ (acc $= 0{,}888$).

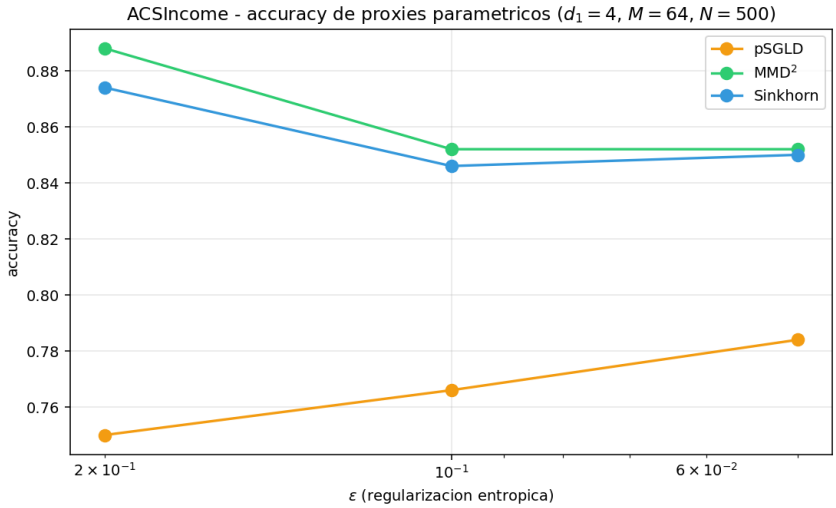

Figura 15: Accuracy de los tres regularizadores paramétricos sobre `ACSIncome` ($d_1 = 4$) en función de la temperatura entrópica ε. MMD^2 y Sinkhorn son indistinguibles a efectos prácticos y dominan a pSGLD en todo el rango.

5.5. Resultados *(C)* —- bridge sobre datos reales

Para que la comparación con ν_T^* sea *apples-to-apples* re-entrenamos los tres proxies con la misma arquitectura $d_1 = 2$ del solver explícito, fijando $\varepsilon = 0{,}05$ (el régimen donde el solver es más fiel al óptimo entrópico). Cada modelo aporta una nube de $M = 64$ neuronas en $\mathbb{R}^5$ que comparamos contra 2 000 muestras categóricas de la ν_T^* exacta del Picard. Las distancias resultantes están en la Tabla 11 y la Figura 16.

método	MMD^2 vs ν_T^*	$\|\mu - \mu^*\|_2$	$\|\Sigma - \Sigma^*\|_F$
pSGLD	**0.156**	0.158	**4.155**
MMD^2	0.179	0.052	4.578
Sinkhorn	0.192	**0.031**	4.540

Cuadro 11: Bridge `ACSIncome` ($d_1 = 2$, $\varepsilon = 0{,}05$): distancia entre la nube empírica de neuronas en $t = T$ de cada proxy y la medida óptima ν_T^* del Theorem 1.4. pSGLD obtiene la MMD^2 más baja — es el proxy más fiel al óptimo entrópico de Daudin-Delarue — mientras que MMD^2/Sinkhorn capturan mejor la media. El ranking en MMD^2 es el mismo que sobre `make_moons` (Tabla 8).

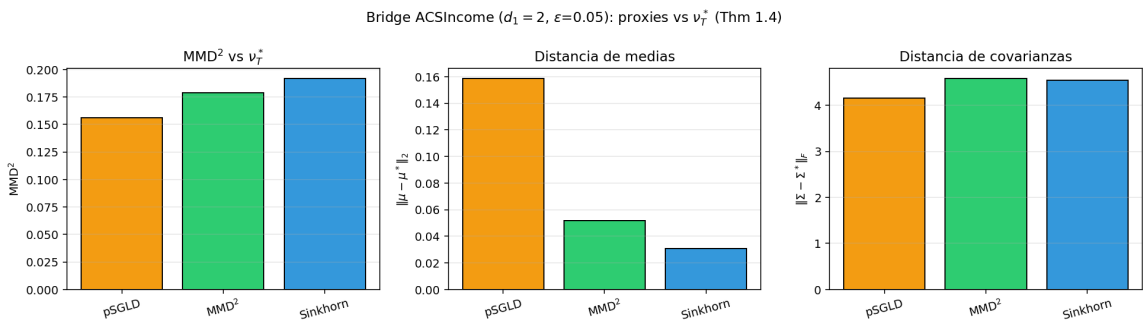

Figura 16: Bridge sobre `ACSIncome`: tres distancias entre la nube de neuronas paramétricas y ν_T^* de Daudin-Delarue. Menor es más cercano a la distribución de Gibbs que el paper caracteriza.

5.6. Otras posibles formulaciones económicas

Marcos económicos alternativos para trabajo posterior. Una vez validado el pipeline sobre `ACSIncome`, otras formulaciones donde cada componente del marco DD admite interpretación económica explícita son:

- **Riesgo sistémico bancario interconectado** [19]: X_t = posición de capital de cada banco; $b(x, a)$ = flujos de préstamo; ν_t = regulación dinámica.
- **HANK / mean-field macro** [20]: γ_t = distribución de riqueza poblacional; ν_t = código fiscal continuo; regularización entrópica $\sim$ coste político de la reforma.
- **Optimización de carteras heterogéneas**: X_t = riqueza individual con horizonte temporal heterogéneo; ν_t = regla de asignación de activos.

Estas formulaciones son matemáticamente más cercanas al espíritu original de DD (control con intervención real), pero requieren modelado económico no trivial y datos no fácilmente disponibles, por lo que se relegan a trabajo posterior.

6. Trabajo futuro

- **Estudio cuantitativo de la propagación del caos.** Variar $M \in \{8, 16, 32, 64, 128, 256, 512, 1024\}$ y, para cada uno, ajustar leyes de escala $\mathrm{MMD}^2(\nu_T^{\mathrm{proxy}}, \nu_T^*) \sim M^{-\alpha}$ por método, como test cuantitativo de la cadena Langevin $\Rightarrow$ Gibbs $\Rightarrow \nu_t^*$ de DD [8, 10] en el setting concreto de esta familia de modelos.
- **Comparación dinámica $t \mapsto \mathrm{MMD}^2(\nu_t^{\mathbf{proxy}}, \nu_t^*)$.** Extender la comparación del bridge a todo el flujo aprovechando la augmentación temporal $a_2^m(t_k) = W_1 \cdot t_k + b_1$, para diagnosticar en qué instantes cada proxy se desvía más del flujo óptimo, no sólo el endpoint $t = T$.
- **Nuevas formas aproximar el regularizador** Diseñar un proxy que aproxime $\mathrm{KL}(\nu_t^{(M)} \| \nu^\infty)$ sin pasar por el grid del solver explícito —por ejemplo estimando la densidad implícita de las partículas con KDE y calculando KL contra ν^∞ por Monte Carlo— como alternativa geométricamente más fiel al funcional original de DD que MMD^2/Sinkhorn.

Referencias

[1] S. Daudin and F. Delarue. Genericity of Polyak-Lojasiewicz inequalities for entropic mean-field neural ODEs. *arXiv:2507.08486*, 2025.

[2] C. Li, C. Chen, D. Carlson, and L. Carin. Preconditioned stochastic gradient Langevin dynamics for deep neural networks. *AAAI Conference on Artificial Intelligence*, 2016.

[3] Y.-A. Ma, T. Chen, and E. B. Fox. A complete recipe for stochastic gradient MCMC. *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[4] M. Girolami and B. Calderhead. Riemann manifold Langevin and Hamiltonian Monte Carlo methods. *Journal of the Royal Statistical Society B*, 73(2):123–214, 2011.

[5] S. Mei, A. Montanari, and P.-M. Nguyen. A mean field view of the landscape of two-layer neural networks. *Proceedings of the National Academy of Sciences*, 115(33):E7665–E7671, 2018.

[6] L. Chizat and F. Bach. On the global convergence of gradient descent for over-parameterized models using optimal transport. *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[7] J. Sirignano and K. Spiliopoulos. Mean field analysis of neural networks: a law of large numbers. *SIAM Journal on Applied Mathematics*, 80(2):725–752, 2020.

[8] K. Hu, Z. Ren, D. Šiška, and Ł. Szpruch. Mean-field Langevin dynamics and energy landscape of neural networks. *Annales de l'Institut Henri Poincaré (B) Probabilités et Statistiques*, 57(4):2043–2065, 2021.

[9] A. Nitanda and T. Suzuki. Stochastic particle gradient descent for infinite ensembles. *arXiv:1712.05438*, 2017.

[10] F. Chen, Z. Ren, and S. Wang. Uniform-in-time propagation of chaos for mean-field Langevin dynamics. *arXiv:2212.03050*, 2022.

[11] G. O. Roberts and R. L. Tweedie. Exponential convergence of Langevin distributions and their discrete approximations. *Bernoulli*, 2(4):341–363, 1996.

[12] A. Durmus and É. Moulines. Nonasymptotic convergence analysis for the unadjusted Langevin algorithm. *The Annals of Applied Probability*, 27(3):1551–1587, 2017.

[13] A. Gretton, K. Borgwardt, M. Rasch, B. Schölkopf, and A. Smola. A kernel two-sample test. *Journal of Machine Learning Research*, 13:723–773, 2012.

[14] D. J. Sutherland, H.-Y. Tung, H. Strathmann, S. De, A. Ramdas, A. Smola, and A. Gretton. Generative models and model criticism via optimized maximum mean discrepancy. *International Conference on Learning Representations (ICLR)*, 2017.

[15] M. Cuturi. Sinkhorn distances: lightspeed computation of optimal transport. *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.

[16] A. Genevay, G. Peyré, and M. Cuturi. Learning generative models with Sinkhorn divergences. *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2018.

[17] J. Feydy, T. Séjourné, F.-X. Vialard, S.-i. Amari, A. Trouvé, and G. Peyré. Interpolating between optimal transport and MMD using Sinkhorn divergences. *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2019.

[18] F. Ding, M. Hardt, J. Miller, and L. Schmidt. Retiring Adult: New datasets for fair machine learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[19] R. Carmona, J.-P. Fouque, and L.-H. Sun. Mean field games and systemic risk. *Communications in Mathematical Sciences*, 13(4):911–933, 2015.

[20] Y. Achdou, J. Han, J.-M. Lasry, P.-L. Lions, and B. Moll. Income and wealth distribution in macroeconomics: a continuous-time approach. *The Review of Economic Studies*, 89(1):45–86, 2022.